Christof Teuscher

ECE 410/510: Hardware for AI and ML, Spring 2026

GPUs + CUDA + CNN/DNN

Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu

teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY

SIMD vs SIMT: Flynn's taxonomy revisited

SIMT (Single Instruction, Multiple Threads) is generally considered a more flexible and programmable evolution of SIMD (Single Instruction, Multiple Data)

Flynn's Taxonomy (1966)

SISD → single-core CPU

SIMD → CPU vectors (SSE/AVX)

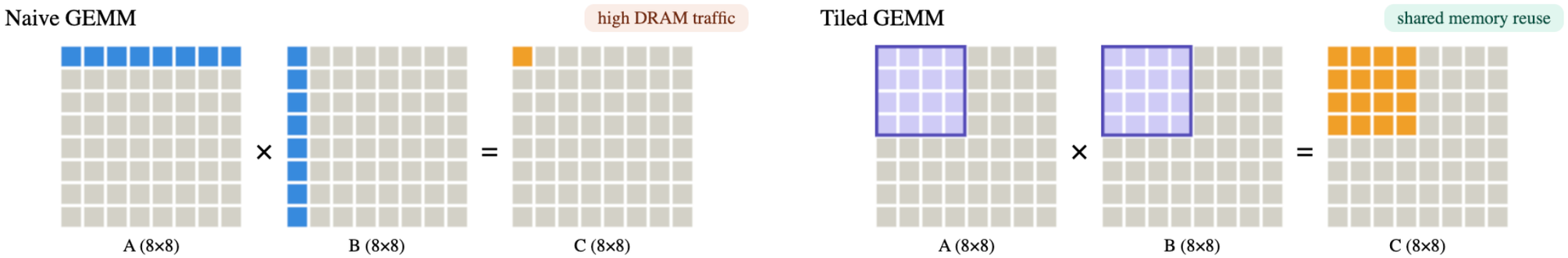
MIMD → multi-core CPU

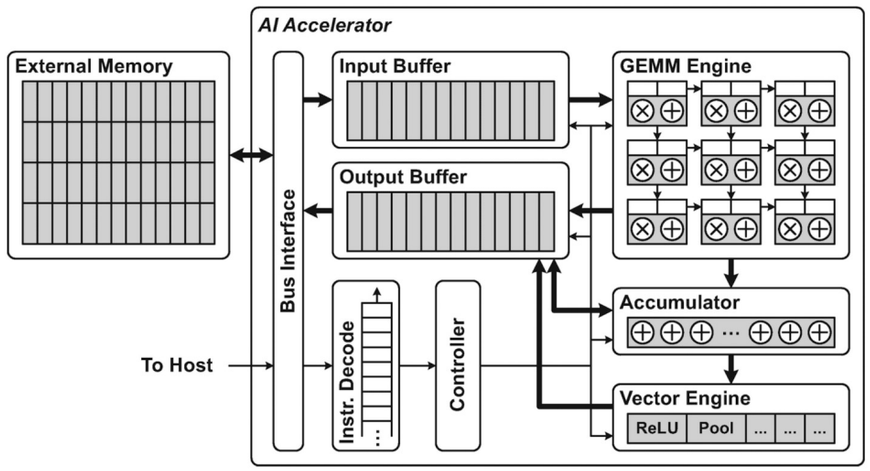
SIMT → GPU (not in Flynn!)

Dimension	SIMD (CPU)	SIMT (GPU)
Parallelism unit	Vector lane (fixed width: 4/8/16)	Thread within warp (32 threads)
Control flow	All lanes must execute identical op	Threads can branch; divergent paths serialized (mask)
Register file	Shared vector regs mapped to scalar	Per-thread private regs (65k 32-bit/SM)
Memory model	Shared cache/L1; no scratchpad control	Shared memory (scratchpad) + L1/L2 + HBM; programmer-managed
Latency tolerance	Out-of-order exec + large caches	Warp switching (0 cycle, zero-overhead); 1000s of active threads hide latency
Scalability	Fixed width; limited by ISA (e.g., Intel AVX-512 = 16x float)	Scales to 1000s of SMs; same code runs on any GPU size

SIMT coined by NVIDIA (Lindholm et al., ISCA 2008). Extends Flynn's SIMD with per-thread PC, stack, and register state

Case study: Naïve vs tiled GEMM





Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

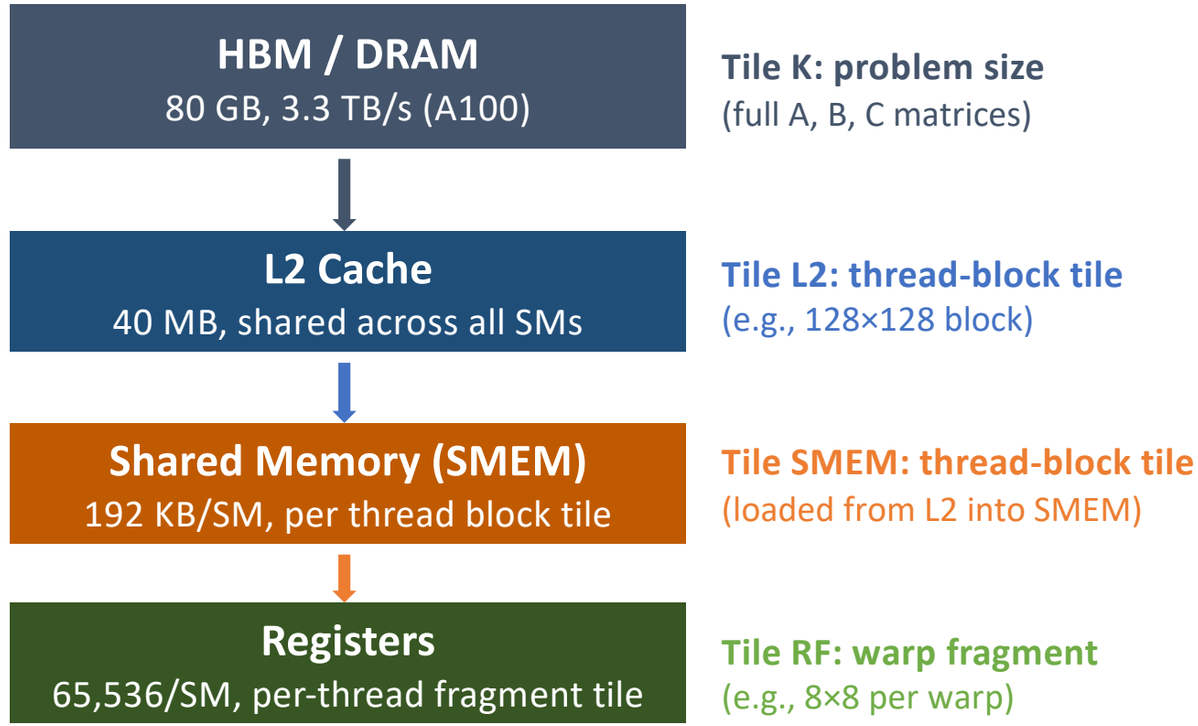
Tiled GEMM on GPU: Memory Hierarchy

Codefest 3 CMAN recap

- GEMM: C = A × B, matrices M×K times K×N
- Naive: $2N^3$ bytes — each element reloaded N times
- Tiled: $2N^2$ bytes — each element loaded exactly once
- **Ratio = $2N^3/2N^2 = N$** (N=32 → 32× less traffic)
- AI: O(1) FLOP/byte → O(N) FLOP/byte

GPUs: three levels of tiling

- CPU model assumed L1/L2 cache holds the tile
- GPU has explicit scratchpad (SMEM) + warp registers
- **Tile size: programmer-controlled, not hardware-invisible, there's no single universally optimal T, it's a multi-constraint optimization. Trade-off between memory sizes, memory traffic, and core occupancy. Empirically determined!**



Christof Teuscher • teuscher@pdx.edu

Tile Size vs GPU Occupancy: The Real Tradeoff

Small Tile (T=16)

Pros:
✓Low SMEM usage per block
✓Many blocks on SM (high occupancy)
✓More warps to hide latency

Cons:
✗Low arithmetic intensity (AI~8)
✗Frequent DRAM reloads
✗Memory-bound: far from roofline peak

→ Memory-bound

Medium Tile (T=32-64)

Pros:
✓Balanced SMEM use
✓Good occupancy (8-16 blocks/SM)
✓AI=16-32 FLOP/B

Cons:
✗May not saturate compute units
✗Warp stalls if SMEM conflicts

→ Sweet spot for many GPUs

Large Tile (T=128+)

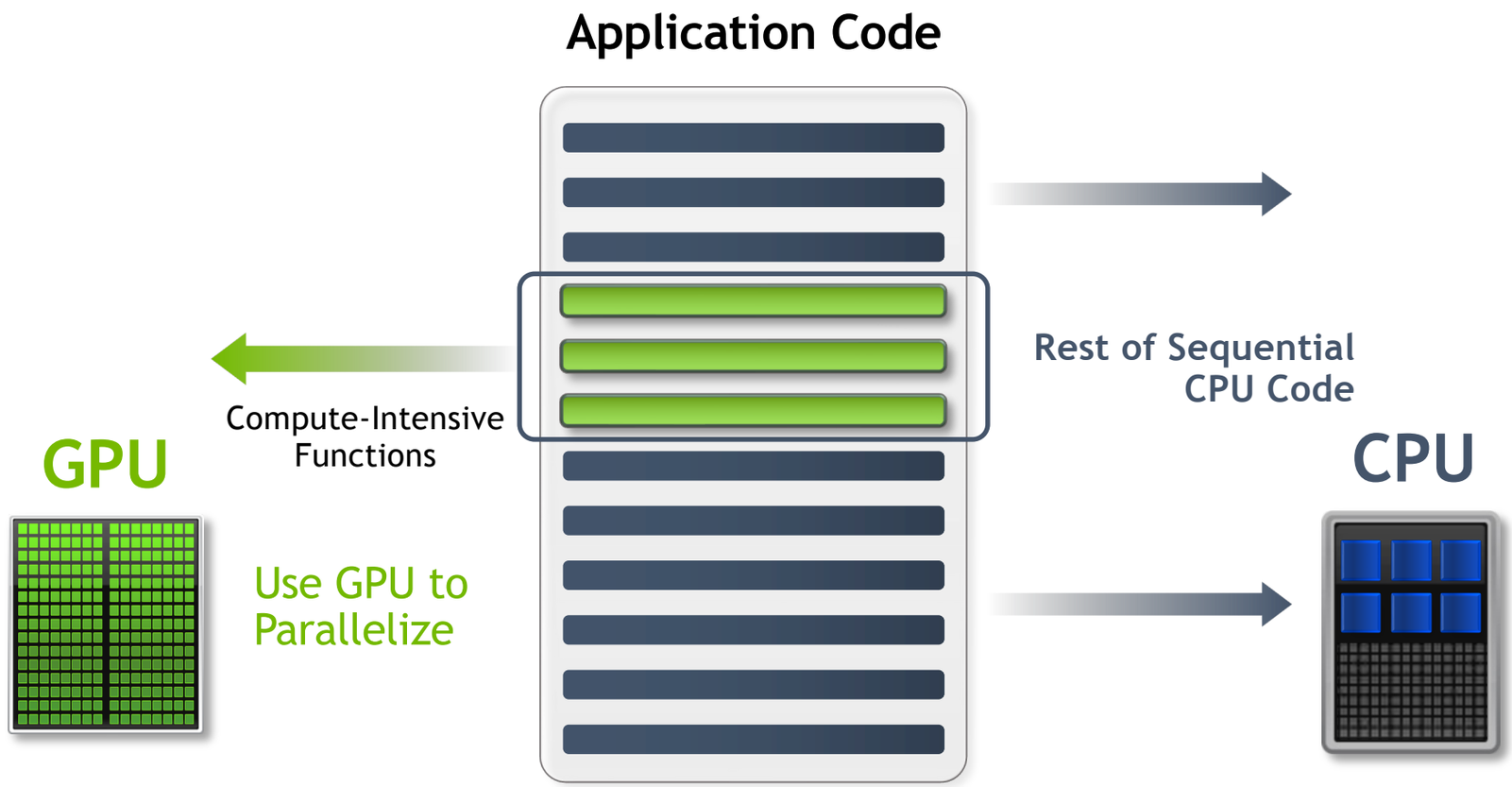
Pros:
✓High AI (~64+ FLOP/B)
✓Fewer DRAM transactions
✓Approaches roofline for FP16

Cons:
✗SMEM caps block count (1-2 blocks/SM)
✗Low occupancy → few warps to hide latency
✗Register pressure spills to L1

→ Compute-bound (ideal if latency hidden)

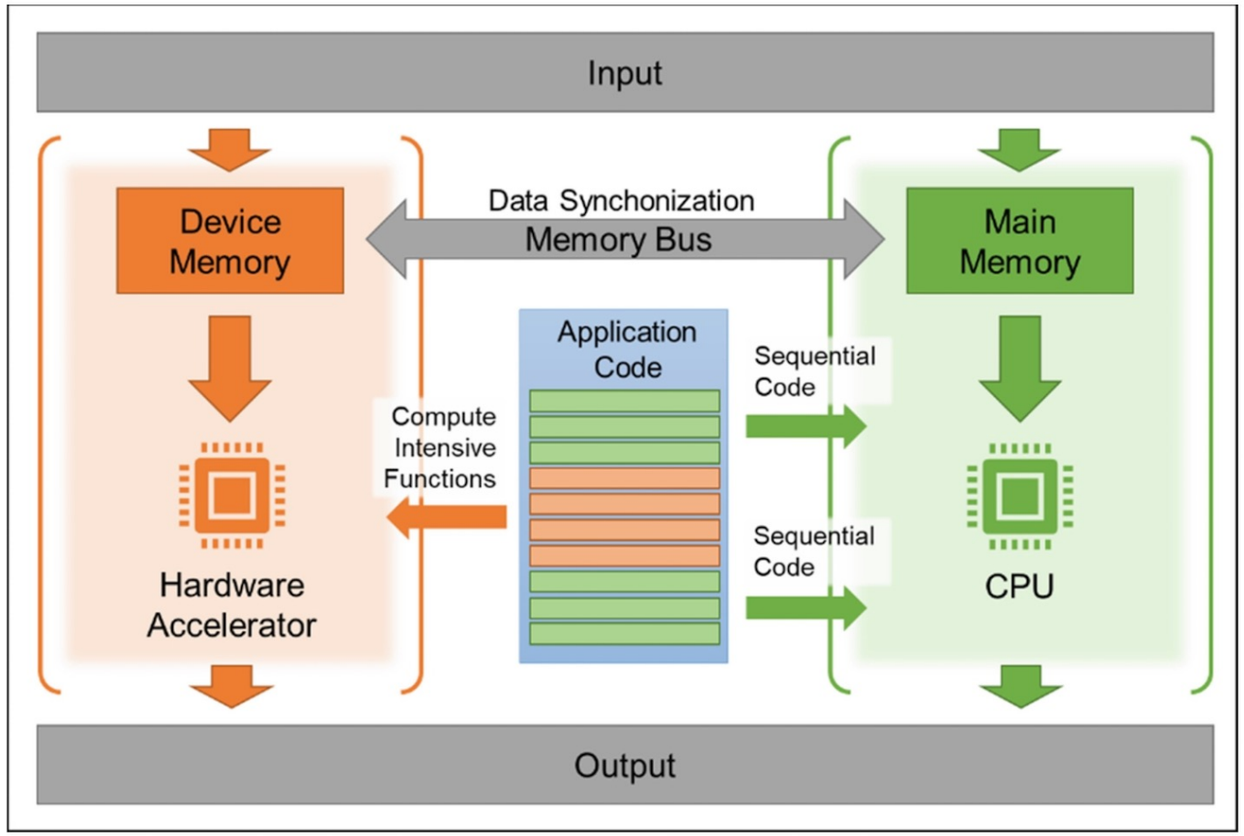
Key insight: Tile size governs the occupancy-intensity tradeoff. Share memory (SMEM) is a finite resource (192 KB/SM on A100). GPU performance requires co-optimizing tile size, warp occupancy, and register pressure, not just maximizing tile size.

General co-processing mechanism



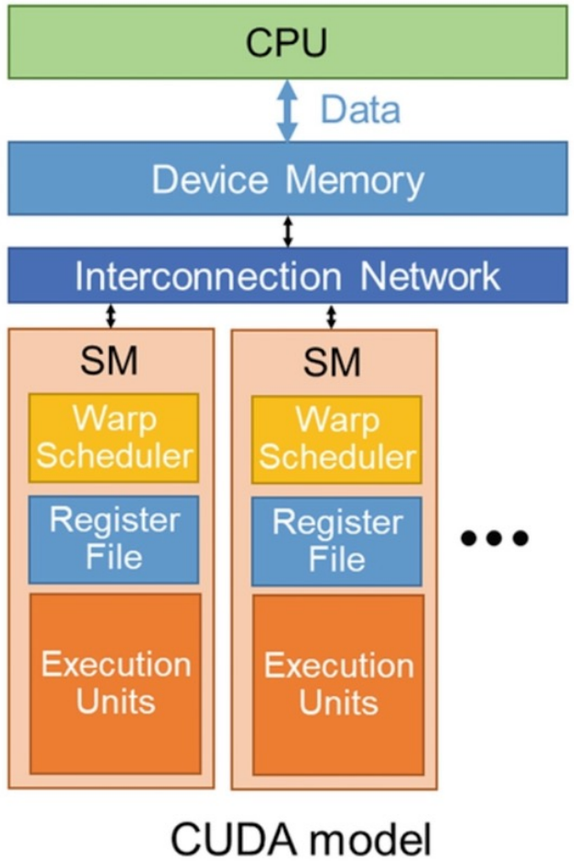
NVIDIA

General co-processing mechanism



Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

GPU/CUDA model



CUDA model

NVIDIA GPU Architecture History	
2025	Blackwell 2.0
2024	Blackwell
2023-2024	Hopper
2022-2024	Ada Lovelace
2020-2024	Ampere
2018-2022	Turing
2017-2020	Volta
2016-2021	Pascal
2014-2019	Maxwell 2.0
2014-2017	Maxwell
2013-2015	Kepler 2.0
2012-2018	Kepler

Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

What is a CUDA Core?

FP:

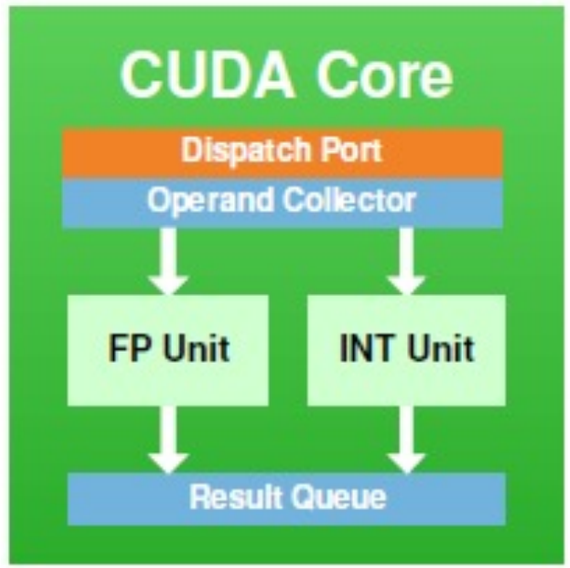
- Addition, subtraction, multiplication, division, square roots.
- CUDA supports both single-precision (FP32) and double-precision (FP64) floating-point operations

INT:

- additions, subtractions, multiplications, comparison, and bitwise operations

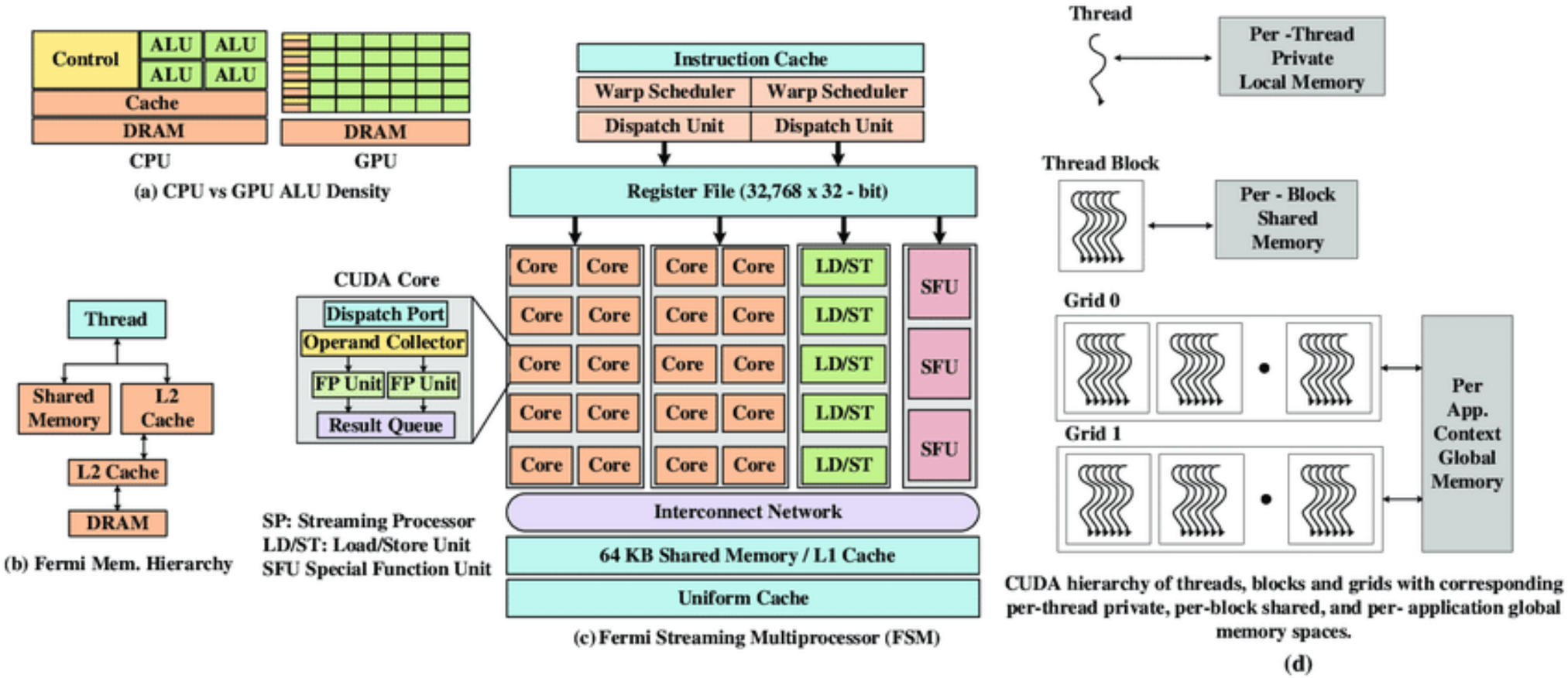
SFU:

- accelerates the execution of specific transcendental and trigonometric functions like sine, cosine, tangent, and exponential calculations



https://www.electronicshub.org/cuda-cores/

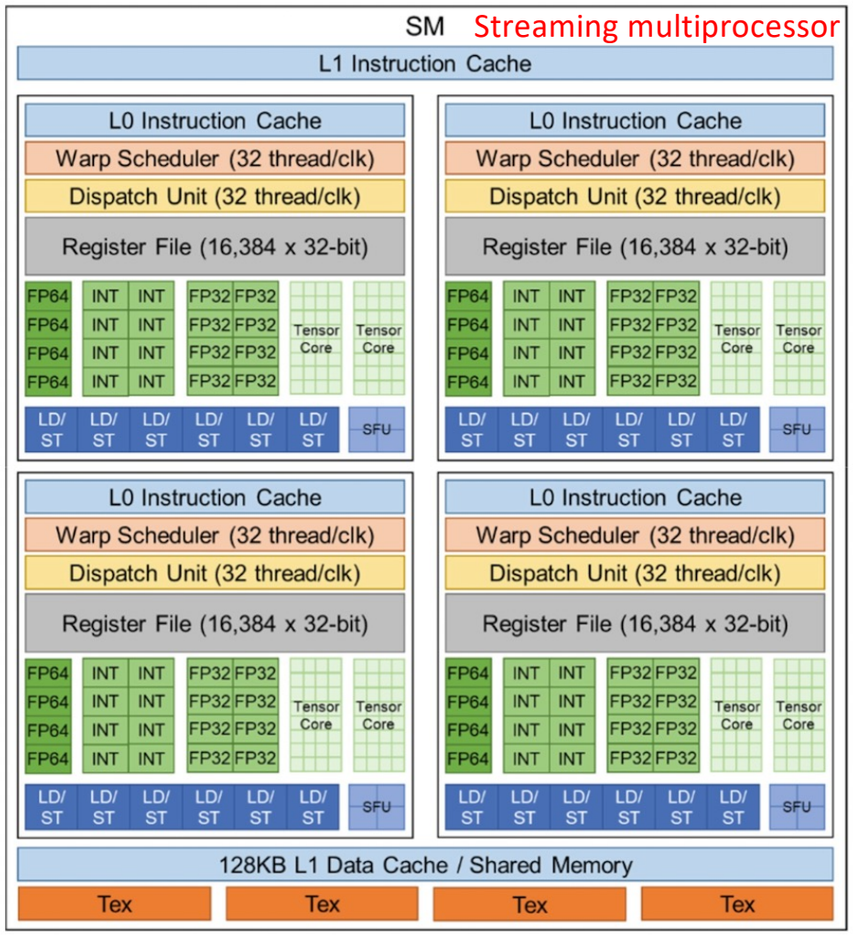
NVIDIA Fermi Microarchitecture



10.1109/ACCESS.2020.3008250

Christof Teuscher • teuscher@pdx.edu

NVIDIA Volta Microarchitecture (2013-2017)



Tex = Texture units
SFU = Special Function Unit

GPU features	Nvidia Tesla P100	Nvidia Tesla V100	Nvidia A100
GPU codename	GP100	GV100	GA100
GPU architecture	Nvidia Pascal	Nvidia Volta	Nvidia Ampere
Compute capability	6.0	7.0	8.0
Threads / warp	32	32	32
Max warps / SM	64	64	64
Max threads / SM	2048	2048	2048
Max thread blocks / SM	32	32	32
Max 32-bit registers / SM	65536	65536	65536
Max registers / block	65536	65536	65536
Max registers / thread	255	255	255
Max thread block size	1024	1024	1024
FP32 cores / SM	64	64	64
Ratio of SM registers to FP32 cores	1024	1024	1024
Shared Memory Size / SM	64 KB	Configurable up to 96 KB	Configurable up to 164 KB

	Supported CUDA Core Precisions								Supported Tensor Core Precisions							
	FP16	FP32	FP64	INT1	INT4	INT8	TF32	BF16	FP16	FP32	FP64	INT1	INT4	INT8	TF32	BF16
Nvidia Tesla P4	No	Yes	Yes	No	No	Yes	No	No	No	No	No	No	No	No	No	No
Nvidia P100	Yes	Yes	Yes	No	No	No	No	No	No	No	No	No	No	No	No	No
Nvidia Volta	Yes	Yes	Yes	No	No	Yes	No	No	Yes	No	No	No	No	No	No	No
Nvidia Turing	Yes	Yes	Yes	No	No	No	No	No	Yes	No	No	Yes	Yes	Yes	No	No
Nvidia A100	Yes	Yes	Yes	No	No	Yes	No	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes	Yes

- Brain Floating Point, BF16
- TensorFloat-32, TF32

Tensor cores (1)

Each Tensor Core operates on a 4x4 matrix and performs the following operation:

D = A×B + C

where A, B, C, and D are 4x4 matrices (Figure 8). The matrix multiply inputs A and B are FP16 matrices, while the accumulation matrices C and D may be FP16 or FP32 matrices (see Figure 8).

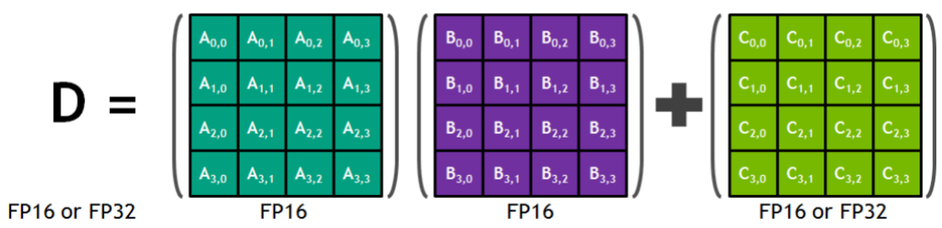


Figure 8. Tensor Core 4x4 Matrix Multiply and Accumulate

Tensor Cores operate on FP16 input data with FP32 accumulation. The FP16 multiply results in a full precision product that is then accumulated using FP32 addition with the other intermediate products for a 4x4x4 matrix multiply (see Figure 9). In practice, Tensor Cores are used to perform much larger 2D or higher dimensional matrix operations, built up from these smaller elements.

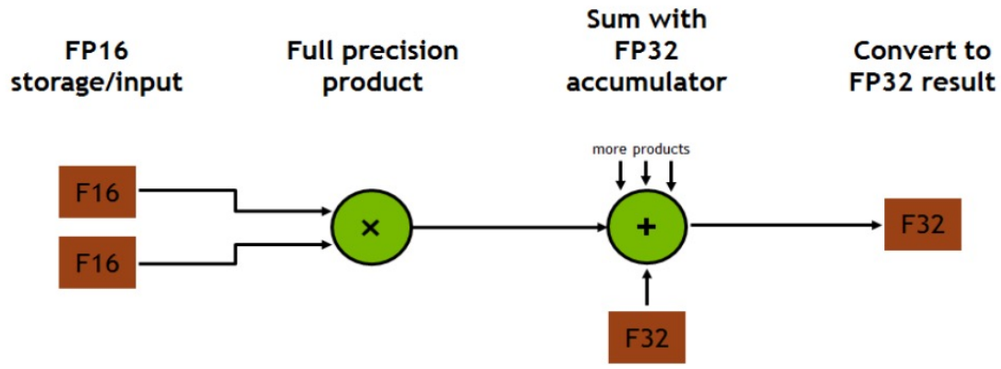
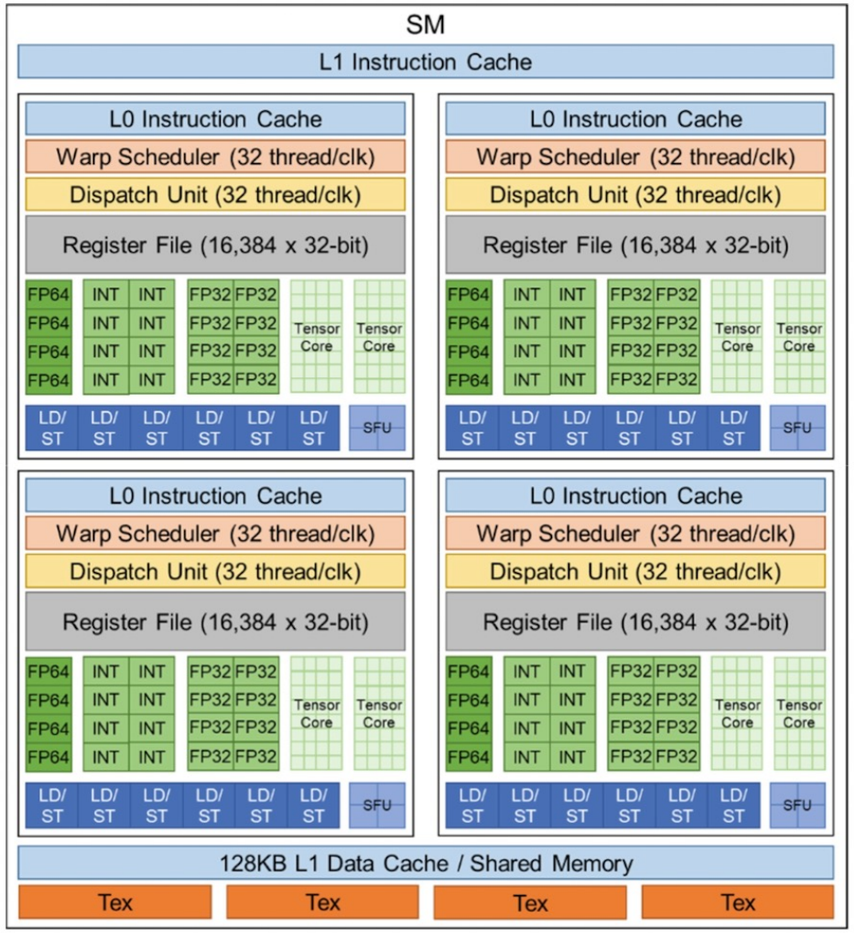


Figure 9. Mixed Precision Multiply and Accumulate in Tensor Core

https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

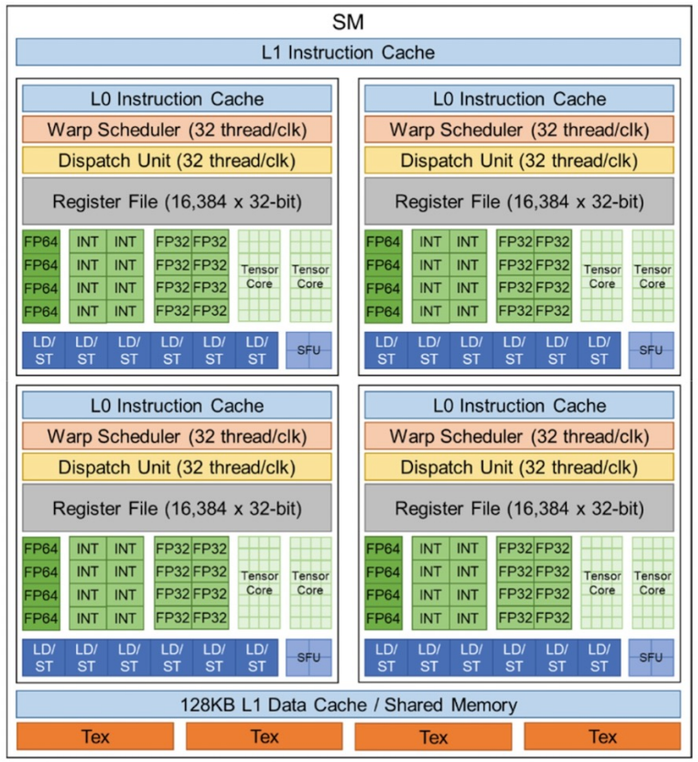
CUDA vs tensor cores



Metric	CUDA Cores	Tensor Cores
Primary Function	General-purpose Computing	Specialized for deep learning matrix computations
Arithmetic Operations	Executes parallel tasks efficiently	Optimized for matrix multiplication
Ideal Applications	Scientific research, machine learning, gaming, and more	Deep learning models, neural network training
Raw Power	Generally, a higher number of cores per GPU	Fewer cores, but highly optimized for specific tasks
Performance Optimization	Efficient for complex algorithms in various fields	Accelerates matrix operations frequently used in AI
>Use Case Suitability	A broad range of applications beyond deep learning	Specific tasks like training large-scale neural networks
Advantage in Deep Learning	Less efficient for matrix-heavy computations	Significant speedups and improved efficiency
Choice Based on Requirements	Suitable for diverse computing needs	Ideal for deep learning projects requiring extensive matrix computations
Future Prospects	Continuous improvements expected	Continuous improvements expected

https://acecloud.ai/resources/blog/cuda-cores-vs-tensor-cores

V100

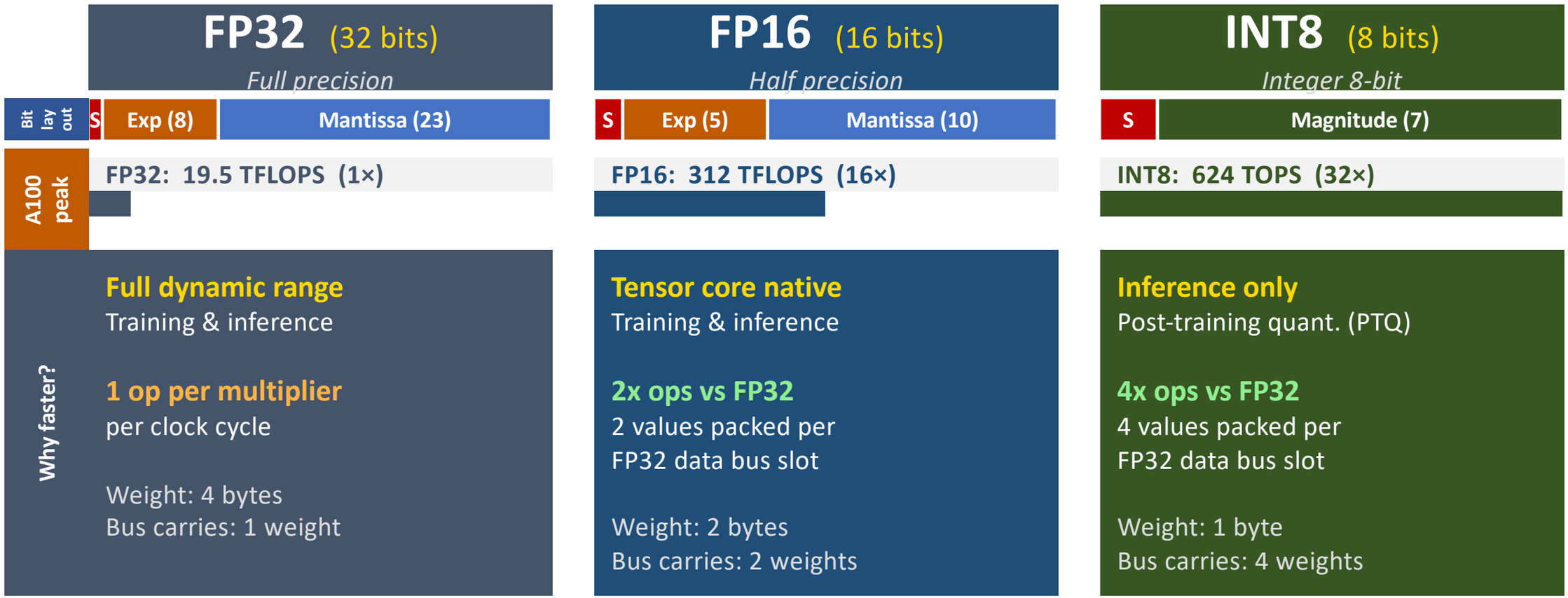


A100

- The NVIDIA V100 features **640 first-generation** Tensor Cores, while the A100 includes **432 third-generation Tensor Cores**.
- Despite the A100 having fewer, more advanced cores, it provides significantly higher performance (up to **312 TFLOPS** TF32) compared to the V100 (**125 TFLOPS**) due to architectural improvements, supporting mixed precision (TF32, FP64, BF16) and other tricks. How is that possible?

Christof Teuscher • teuscher@pdx.edu

Why reduced precision boosts throughput

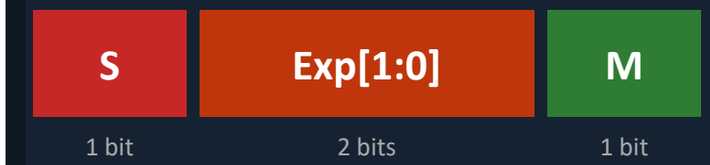


Key insight: Halving precision doubles the number of operands that fit in a fixed data bus, a register, or a tensor-core tile, giving 2x or 4x more MACs per cycle at the same silicon area and power budget.

FP4 — 4-Bit Floating Point Format (E2M1)

Blackwell / B200 GPU

BIT FORMAT: E2M1 — the FP4 encoding



S = Sign bit (0: +, 1: −) | Exp = Exponent (bias-1) | M = Mantissa
E2M1 = 2 exponent bits + 1 mantissa bit → 4 bits total

All 16 representable values (−6 to +6):

0	±0.5	±1	±1.5	±2	±3	±4	±6
−0.5	−1	−1.5	−2	−3	−4	−6	NaN

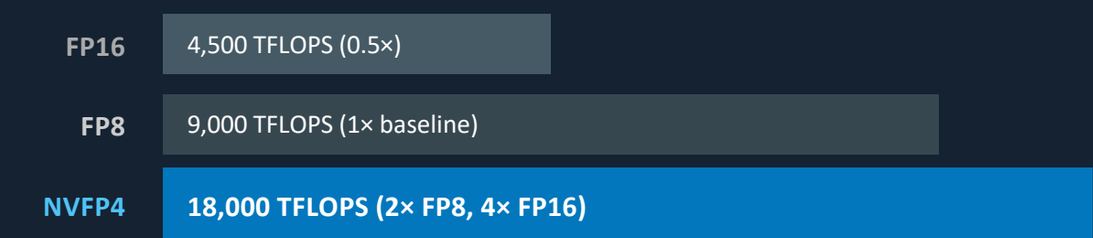
Only 16 distinct values! Compare: FP16 has 65,536 values, FP8 has 256 values.

The precision problem:
With only 16 values, a single scale factor per tensor loses accuracy. Different weight layers span very different value ranges, one global scale cannot represent them all precisely.
Solution → NVFP4: assign one FP8 scale factor per 16-value block (see next slide).

WHY FP4 — Precision vs. Throughput Trade-off

Format	Bits	Exp	Man	Values
FP16	16	5	10	65,536
FP8 E4M3	8	4	3	256
FP4 E2M1	4	2	1	16

B200 GPU THROUGHPUT — Dense Tensor Core ops/s



Abbreviations: FP4 = 4-bit Floating Point | FP8 = 8-bit FP | FP16 = 16-bit FP | E2M1 = 2 Exponent + 1 Mantissa bits | TFLOPS = Tera Floating-Point Ops/sec

Key insight: FP4 holds only 16 distinct values. Its narrow range requires micro-block scaling to maintain model accuracy. Next slide explains NVFP4.

NVFP4 — NVIDIA's Micro-Block Scaled 4-Bit Format

Blackwell / B200 GPU

MICRO-BLOCK SCALING — How NVFP4 extends FP4 range

FP4 has only 16 values. To represent the full range of a neural network weight tensor, NVFP4 groups values into **blocks of 16** and assigns each block its own shared scaling factor.

Block of 16 FP4 values (E2M1 each):

↓

FP8 E4M3 scale
1 scale shared ×16 values

× FP32 tensor-level scale

real_value = FP4_val × FP8_block_scale × FP32_tensor_scale

NVFP4 (NVIDIA, Blackwell): 16-value blocks, FP8 E4M3 scale
MXFP4 (open standard): 32-value blocks, E8M0 scale
Effective precision: ~4.5 bits/value | Memory: 3.5× smaller vs FP16

NVFP4 BENEFITS — Why it matters for AI inference

2× throughput
vs FP8 on B200 Tensor Cores: 18,000 vs 9,000 TFLOPS. FP4 ops fit twice as many MACs per clock cycle.

2× memory
Half the weight size vs FP8 → twice as many model parameters fit in HBM3e per GPU.

≤1% accuracy loss
PTQ from FP8: DeepSeek-R1 benchmark within 1% of FP8 baseline.

Target workloads
LLM inference · weight-only or W4A4 · PTQ via TensorRT-LLM

Abbreviations explained:
NVFP4 = NVIDIA's 4-bit FP with micro-block scaling (Blackwell-native)
MXFP4 = Microscaling FP4 (open MX standard, 32-value blocks)
PTQ = Post-Training Quantization | LLM = Large Language Model
W4A4 = 4-bit Weights & 4-bit Activations | HBM = High Bandwidth Memory

Abbreviations: FP4/FP8/FP16 = 4/8/16-bit Floating Point | MAC = Multiply-Accumulate | TFLOPS = Tera FP Ops/sec | E4M3 = 4 Exp + 3 Mantissa bits

Key insight: NVFP4 = FP4 values × FP8 block scale × FP32 tensor scale = 3 levels of precision, ~4.5 effective bits, 2× throughput vs FP8 on Blackwell.

NVFP4 Worked Example — Step by Step with Real Numbers

Blackwell / B200 GPU

STEP 1 — Quantize 16 weights into one FP4 block

Imagine a layer with 16 weights ranging from −3.6 to +2.8. We store them as NVFP4.

Original FP16 weights (sample):

+2.8	+1.9	+1.3	+0.7	+0.4	+0.2	+0.1	−0.1
−0.5	−0.9	−1.4	−2.0	−2.6	−3.1	−3.4	−3.6

Compute FP8 block scale:

max|w| = 3.6 FP4 E2M1 max value = 6.0

block_scale = 3.6 / 6.0 = 0.600 → stored as FP8 E4M3

Quantized FP4 codes (divide by scale, round to nearest):

+4	+3	+2	+1	+0.5	0	0	0
−0.5	−1	−2	−3	−4	−4	−4	−6

Stored per block: 16 × 4 bits = 64 bits of FP4 codes
+ 1 × 8 bits of FP8 block_scale → 72 bits total (4.5 bits/weight)
vs FP16: 16 × 16 bits = 256 bits → 3.6× memory saving

STEP 2 — Dequantize: reconstruct original values

reconstructed = FP4_code × block_scale (0.600)

FP4 code	× scale	Reconstructed	Original FP16	Error
+4	×0.600	2.400	+2.8	−0.40
+3	×0.600	1.800	+1.9	−0.10
+1	×0.600	0.600	+0.7	−0.10
0	×0.600	0.000	+0.1	−0.10
−0.5	×0.600	−0.300	−0.5	−0.20
−2	×0.600	−1.200	−1.4	−0.20
−4	×0.600	−2.400	−3.1	−0.70
−6	×0.600	−3.600	−3.6	0.00

… (8 more weights, same process) …

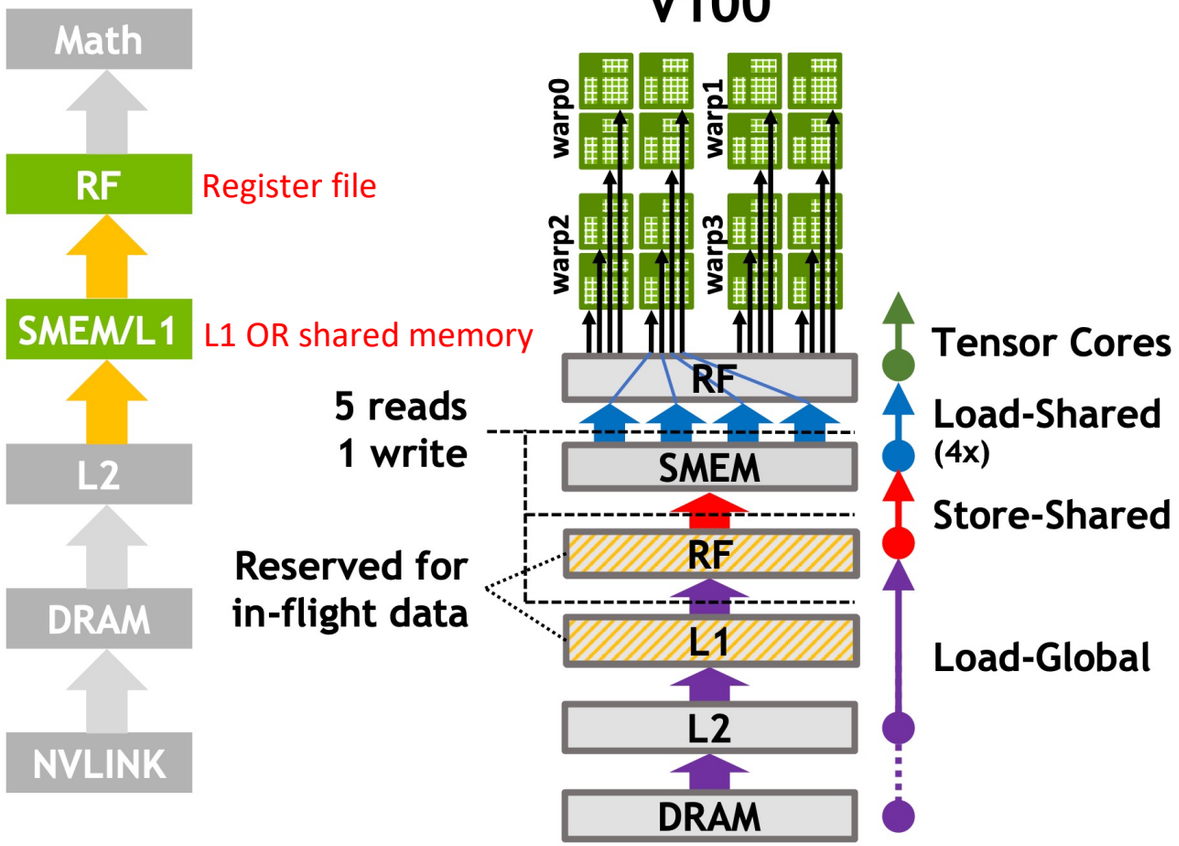
Max error = 0.40 = 11% of range (3.6+3.6=7.2) | Most errors < 5%

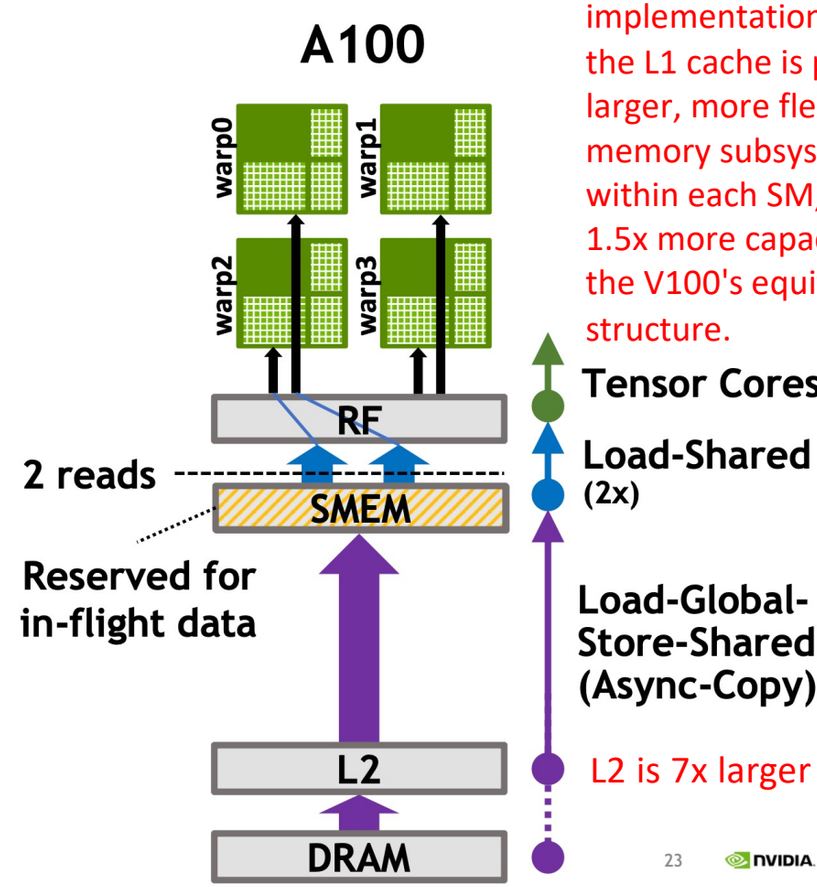
Abbreviations: FP4 code = nearest representable FP4 value | block_scale = FP8 E4M3 (8-bit) | Dequantize = multiply back by scale to recover original units | **Note:** FP32 tensor-level scale (see slide 17) assumed = 1.0 in this example for clarity

Key insight: The block scale absorbs the range (÷6.0 maps −3.6…+3.6 onto −6…+6). Only one 8-bit number per 16 weights, i.e., 4.5 bits/weight effective, 3.6× smaller than FP16.

A100 data movement efficiency

3x SMEM/L1 bandwidth, 2x in-flight capacity





The A100 features an improved implementation where the L1 cache is part of a larger, more flexible memory subsystem within each SM, offering 1.5x more capacity than the V100's equivalent structure.

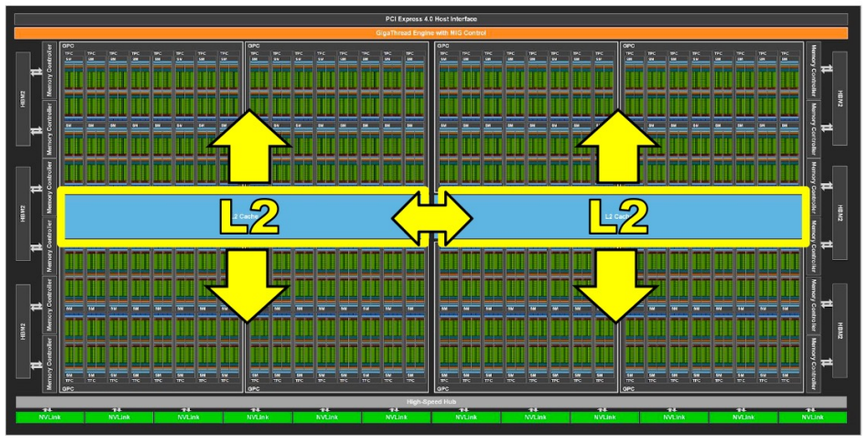
L2 is 7x larger in A100!

23 NVIDIA

in-flight data = data that is actively being processed or transferred within the system but hasn't yet reached its final destination.

A100 data movement efficiency

Split L2 with hierarchical crossbar - 2.3x increase in bandwidth over V100, lower latency

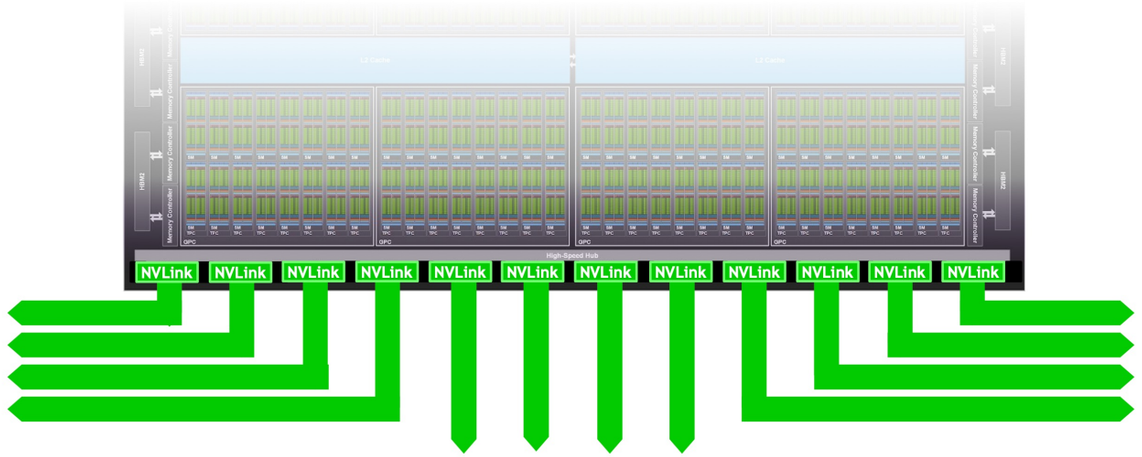


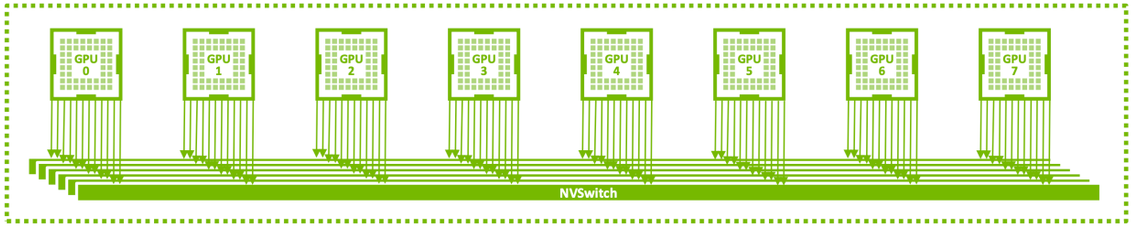
Third Generation NVLink

50 Gbit/sec per signal pair

12 links, 25 GB/s in/out, 600 GB/s total

2x vs. V100





Hardware consistency

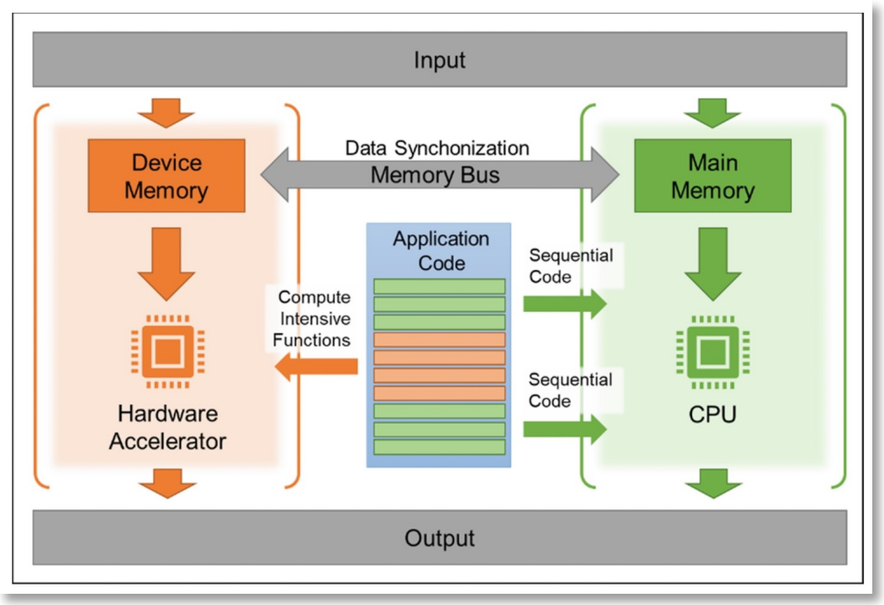
CUDA

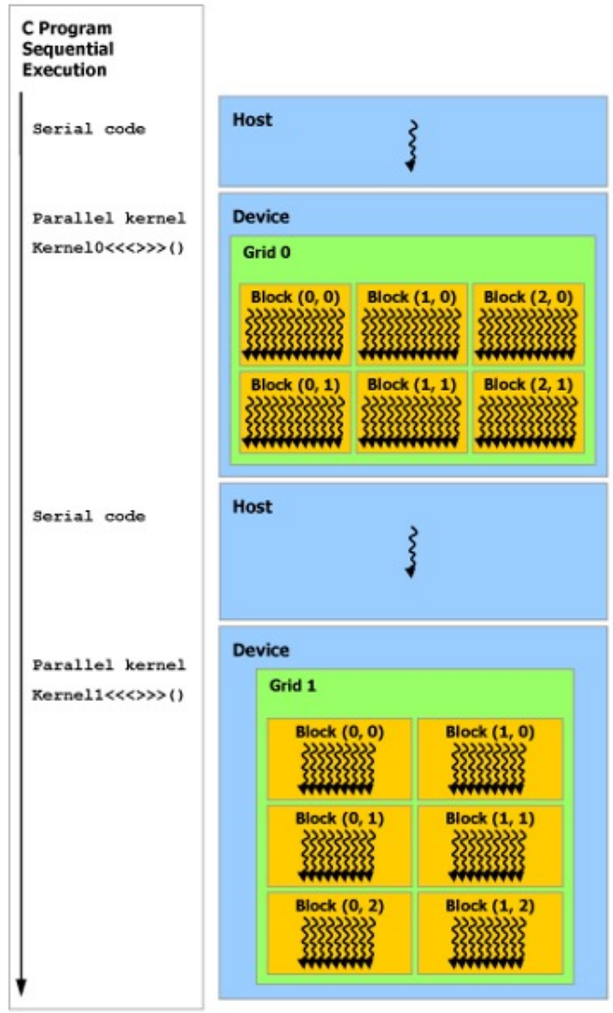
- **CUDA** (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA.
- It enables developers to use NVIDIA graphics processing units (GPUs) for general-purpose computing, a practice known as **GPGPU** (General-Purpose computing on Graphics Processing Units).
- CUDA programs to be written without knowing the exact hardware configuration in advance.

- Good resource: https://docs.nvidia.com/cuda/cuda-c-programming-guide

Programming and execution model

A GPU is built around an **array of Streaming Multiprocessors (SMs). A multithreaded program partitioned into blocks of threads that execute independently from each other**, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.





CUDA programming model assumes that the **CUDA threads execute on a physically separate device** that operates as a coprocessor to the host running the C++ program.

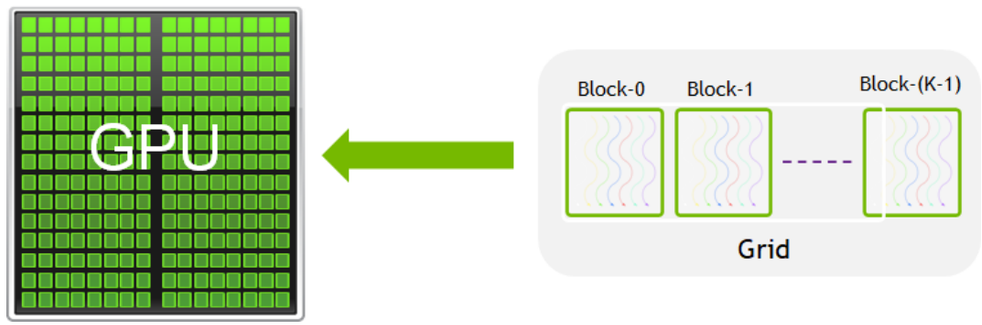
Serial code executes on the host while parallel code executes on the device.

teuscher Lab
teuscher-lab.com

Portland State UNIVERSITY

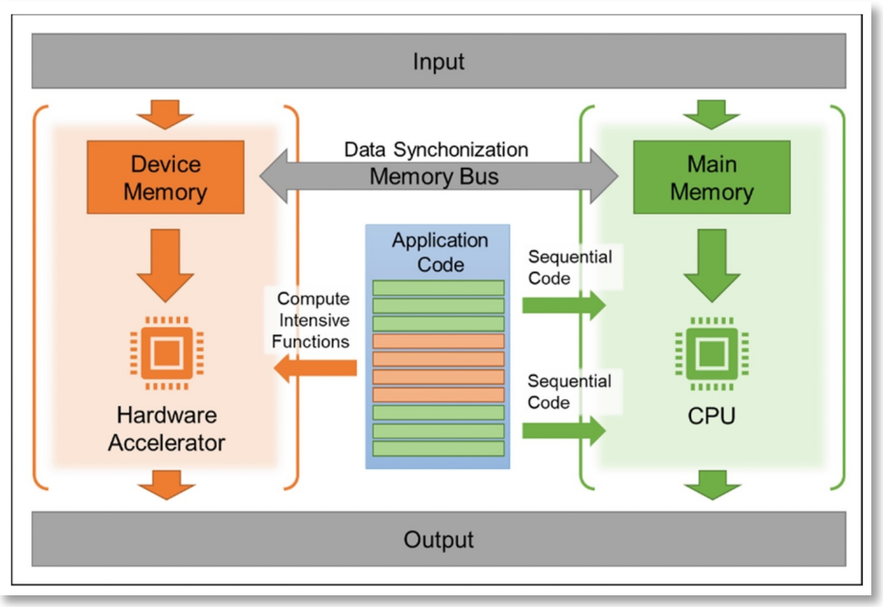
CUDA terminology

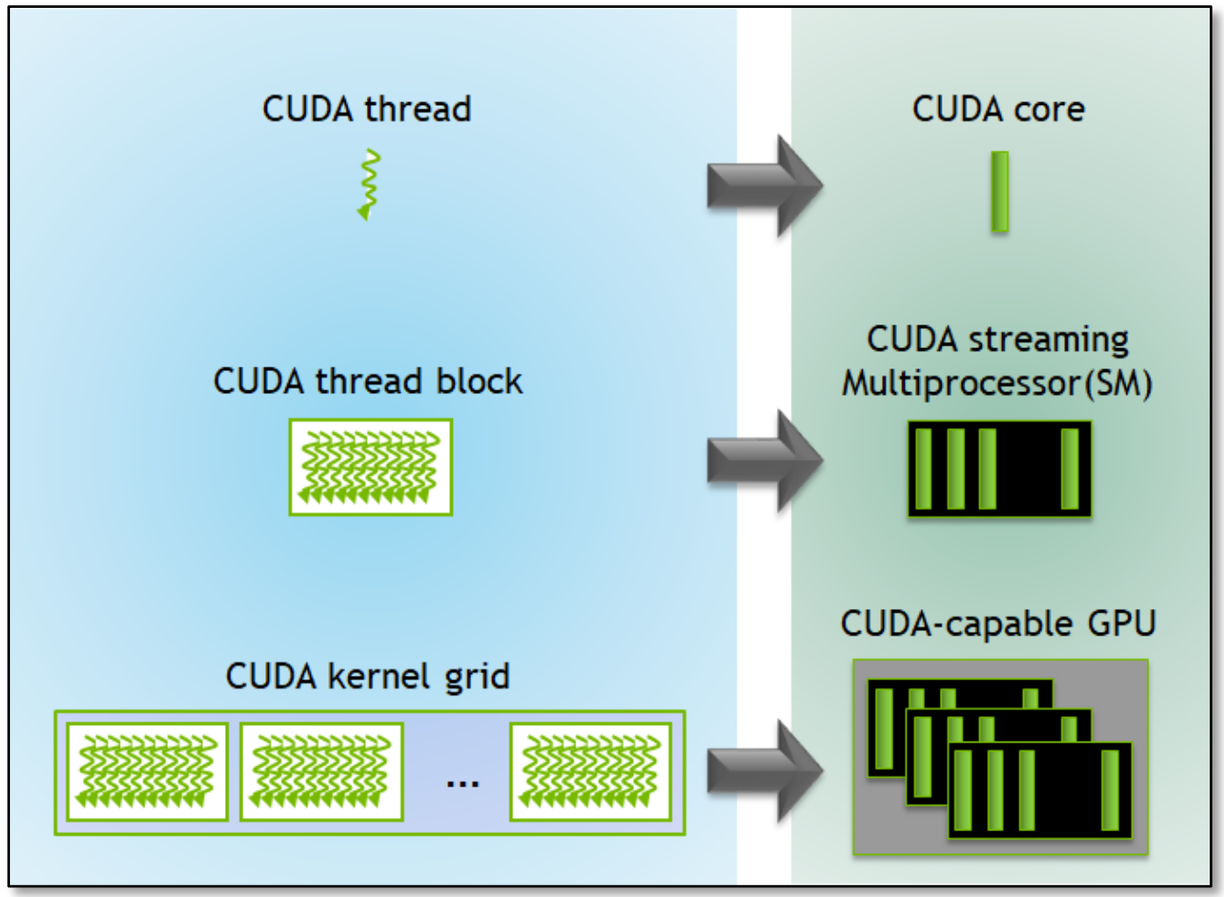
- A **warp** in CUDA is a fundamental execution unit consisting of 32 threads that execute in lockstep. All 32 threads in a warp execute the same instruction at the same time on different data (**SIMD-ish**: Single Instruction, Multiple Data).
- A **CUDA kernel** is a function that runs on the GPU.
- A **block** in CUDA (also called a **thread block**) is a fundamental organizational unit of threads that execute a kernel function.
- The number **of threads per block** is limited (typically 1024 threads in modern GPUs), as they must reside on the same streaming multiprocessor (**SM**).
- **Thread blocks are composed of multiple warps**. For example, a block of 128 threads would consist of 4 warps.
- In CUDA programming, a **kernel grid** is the highest-level organization of threads when executing a CUDA kernel.



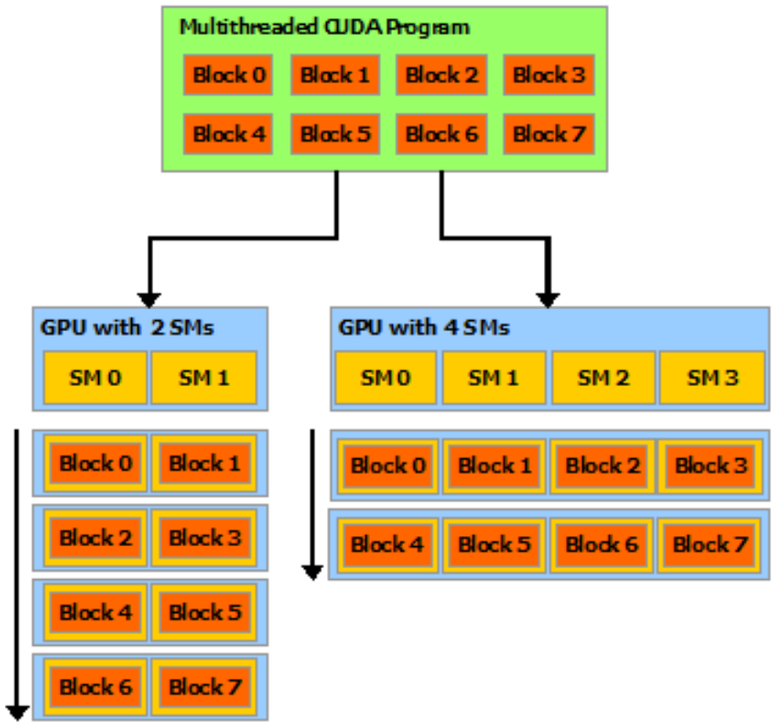
Programming and execution model

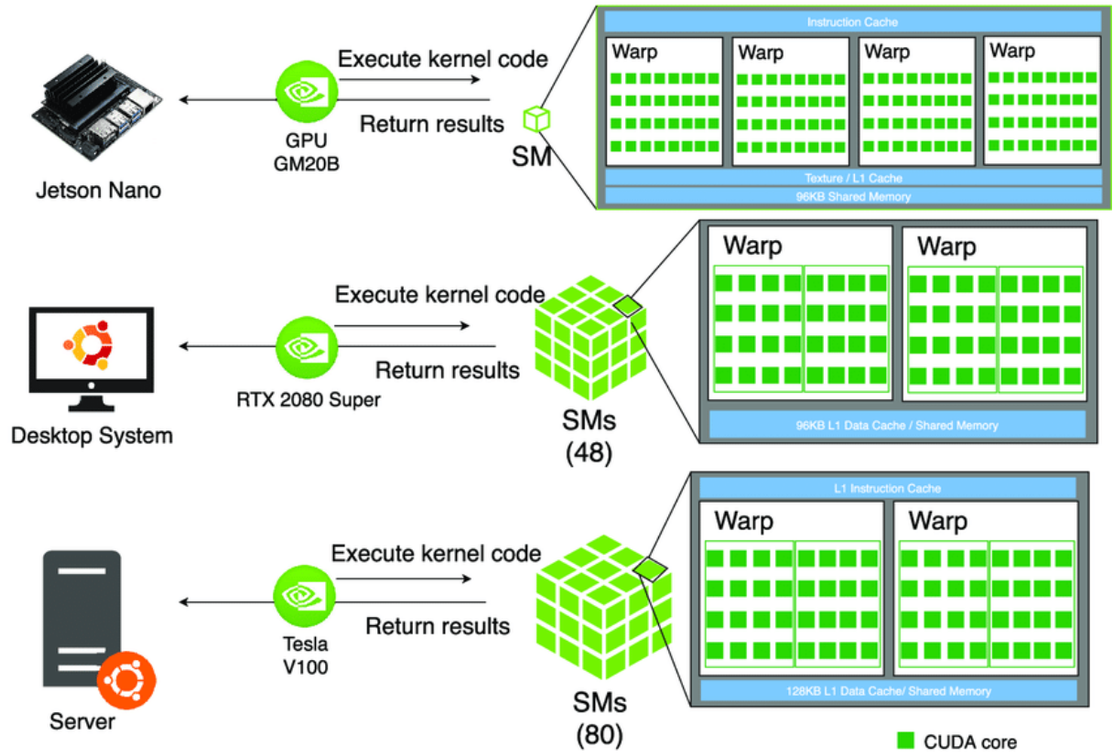
A GPU is built around an **array of Streaming Multiprocessors (SMs). A multithreaded program is partitioned into blocks of threads that execute independently from each other**, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.





Automatic scalability



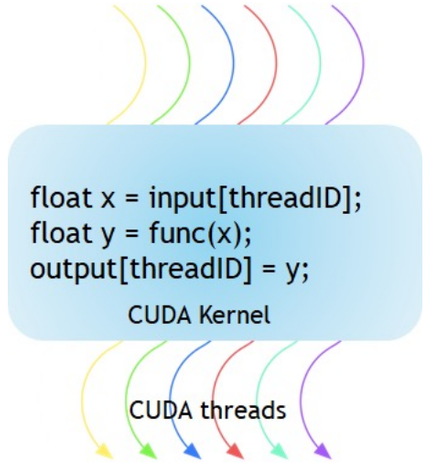


A GPU is built around an **array of Streaming Multiprocessors (SMs). A multithreaded program is partitioned into blocks of threads that execute independently from each other**, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.

Kernels (1)

- CUDA C++ extends C++ by allowing the programmer to define C++ functions, called **kernels**, that, when called, **are executed *N* times in parallel by *N* different CUDA threads, as opposed to only once like regular C++ functions**.
- A kernel is defined using the **__global__ declaration specifier** and the number of CUDA threads that execute that kernel for a given kernel call is specified using a new **<<<...>>> execution configuration** syntax.
- Each thread that executes the kernel is given a unique thread ID that is accessible within the kernel through built-in variables.

```
#include <iostream>

__global__ void kernel( void ) {
}

int main( void ) {
    kernel<<<1,1>>>();
    printf( "Hello, World!\n" );
    return 0;
}
```



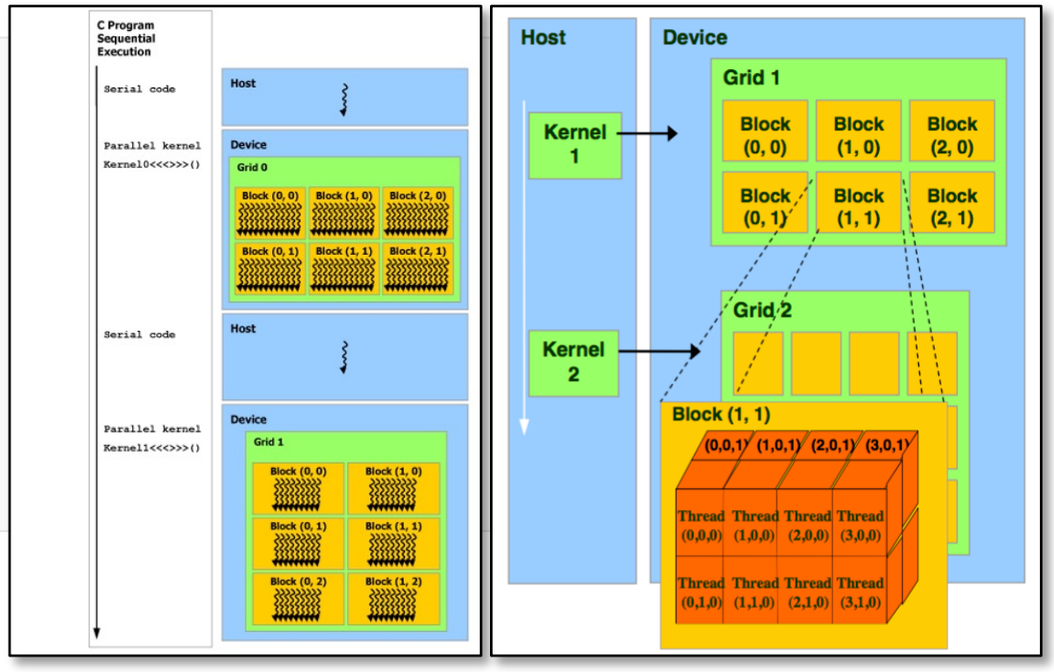
https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model

Kernels (2)

- As an illustration, the following sample code, using the built-in variable threadIdx, adds two vectors A and B of size N and stores the result into vector C.
- Single Instruction, Multiple Threads (SIMT)

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

int main()
{
    ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(A, B, C);
    ...
}
```

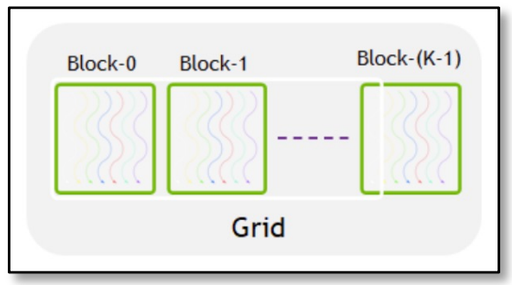
Here, each of the *N* threads that execute `VecAdd()` performs one pair-wise addition.



Kernels (3)

- `kernel<<M,T>>`
- The kernel launches with a grid of M thread blocks. Each thread block has T parallel threads.

- The information between the triple chevrons is the execution configuration, which dictates how many device threads execute the kernel in parallel.
- In the CUDA programming model, we speak of **launching a kernel with a grid of thread blocks**.
- The first argument in the execution configuration specifies the number of thread blocks M in the grid, and the second specifies the number of threads T in a thread block.

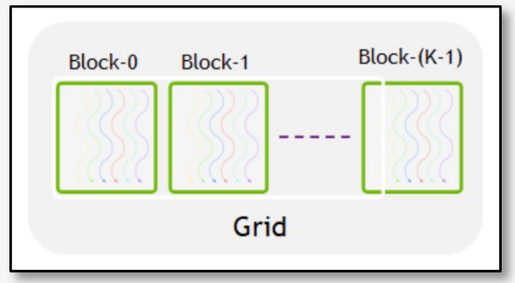


https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model

Kernels (4)

The CUDA program for adding two matrices below shows multi-dimensional `blockIdx` and `threadIdx` and other variables like `blockDim`. In the example below, a 2D block is chosen for ease of indexing and each block has 256 threads with 16 each in x and y-direction. The total number of blocks are computed using the data size divided by the size of each block.

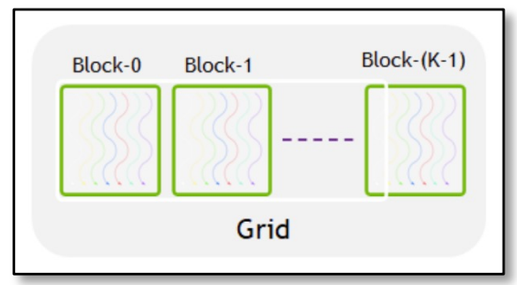
```
// Kernel - Adding two matrices MatA and MatB
__global__ void MatAdd(float MatA[N][N], float MatB[N][N],
float MatC[N][N])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < N && j < N)
        MatC[i][j] = MatA[i][j] + MatB[i][j];
}

int main()
{
    ...
    // Matrix addition kernel launch from host code
    dim3 threadsPerBlock(16, 16);
    dim3 numBlocks((N + threadsPerBlock.x -1) / threadsPerBlock.x, (N+threadsPerBlock.y -1) / threadsPerBlock.y);
    MatAdd<<<numBlocks, threadsPerBlock>>>(MatA, MatB, MatC);
    ...
}
```



https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model

Kernels (4)

What happens if you ask for more blocks than are physically available on the GPU?

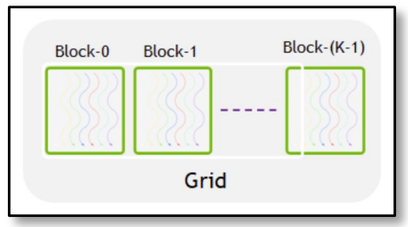


https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model

Kernels (4)

What happens if you ask for more blocks than are physically available on the GPU?

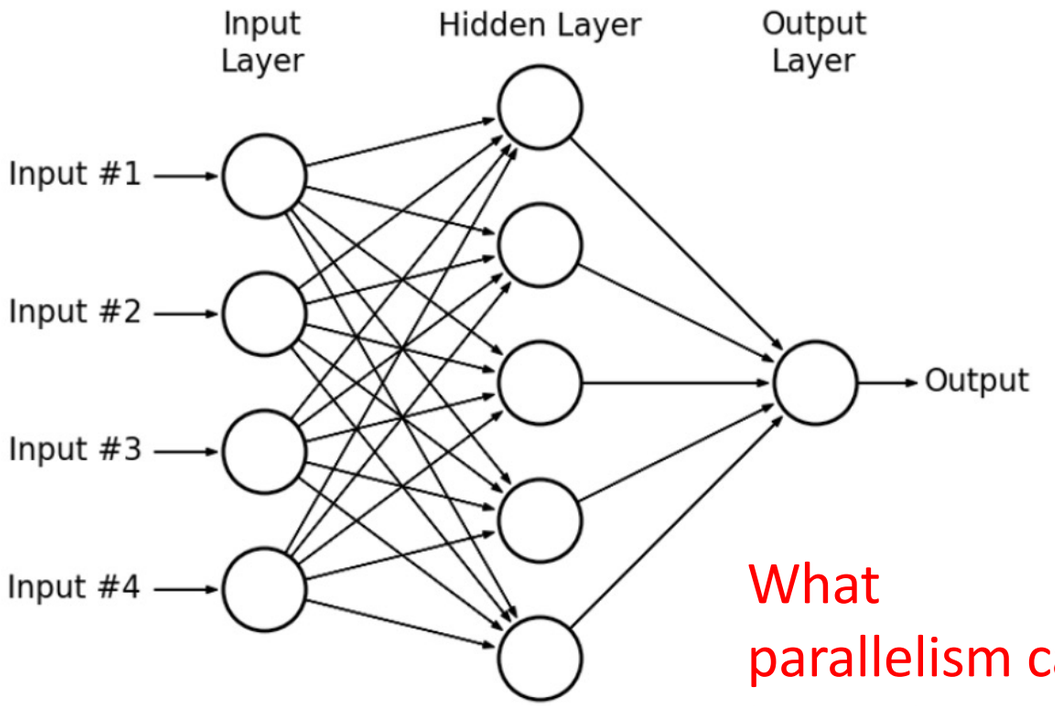
- The GPU doesn't reject your kernel launch. Instead, it schedules the execution of blocks over time. This is a key aspect of CUDA's execution model.
- The GPU hardware scheduler automatically queues the blocks and executes them as resources become available.
- Blocks waiting in the queue don't make progress until scheduled
- The total execution time will increase as blocks must be processed sequentially
- Memory for all blocks must still be allocated, potentially creating memory pressure.
- GPUs have a metric called "**occupancy**" which represents how many threads can be active simultaneously relative to the theoretical maximum. The scheduler tries to maximize occupancy based on resource availability.
- CUDA programs can be written without knowing the exact hardware configuration.



https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model

How to map a neural network on a GPU?

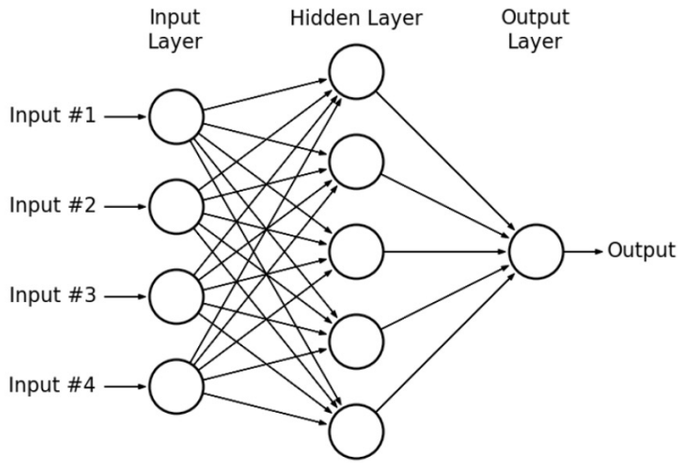
Multi-layer perceptron (MLP)

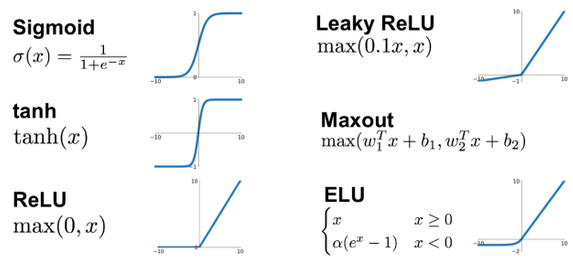


What parallelism can be exploited?

Multi-layer perceptron (MLP)

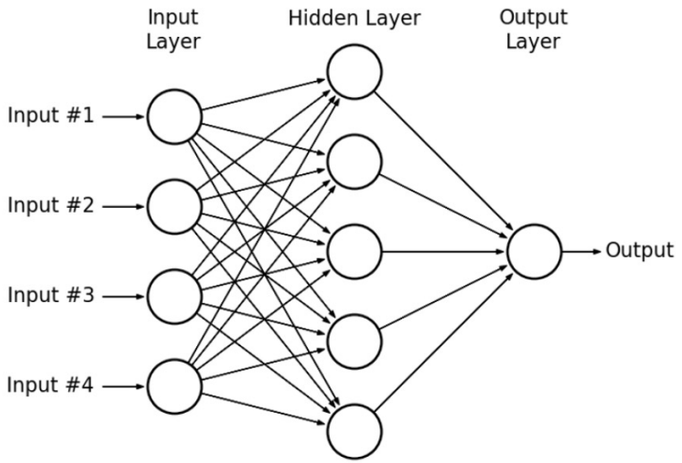
- **Data parallelism**: You can process multiple training examples simultaneously. Since each example is processed independently through the forward pass, you can distribute batches across different computing units.
- **Model parallelism**: The network itself can be partitioned, with different parts of the model assigned to different processors. This works particularly well for very large networks.
- **Layer-wise parallelism**: Within each layer, the computations can be parallelized since neurons in the same layer operate independently.

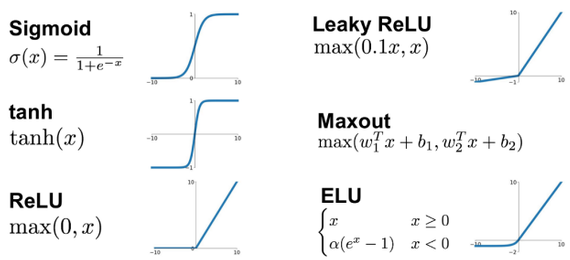




Multi-layer perceptron (MLP)

- **Matrix multiplications**: The core operation in MLPs (Wx + b) is highly parallelizable. Each output neuron's computation is independent.
- **Activation functions**: These are applied element-wise and can be computed in parallel across all neurons.
- **Backpropagation**: During training, gradient computations can be parallelized both within and across layers.



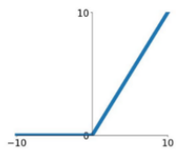


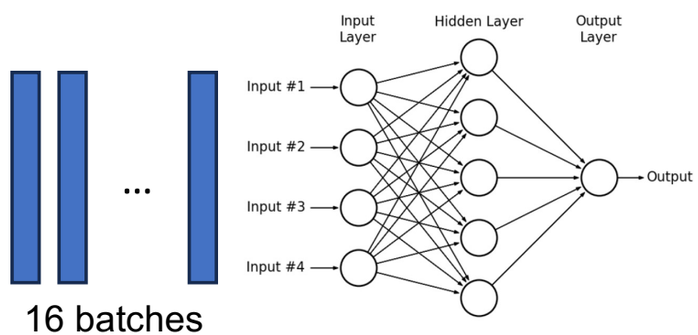
CUDA MLP (1)

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>
#include <time.h>

// Dimensions of our network
#define INPUT_SIZE 4
#define HIDDEN_SIZE 5
#define OUTPUT_SIZE 1
#define BATCH_SIZE 16  // Process 16 samples at once

// Activation function (ReLU)
__device__ float relu(float x) {
    return x > 0 ? x : 0;
}
```

ReLU

$$\max(0, x)$$





Christof Teuscher • teuscher@pdx.edu

CUDA C++ Language Extensions

7.1. Function Execution Space Specifiers

Function execution space specifiers denote whether a function executes on the host or on the device and whether it is callable from the host or from the device.

7.1.1. __global__

The `__global__` execution space specifier declares a function as being a kernel. Such a function is:

- Executed on the device,
- Callable from the host,
- Callable from the device for devices of compute capability 5.0 or higher (see CUDA Dynamic Parallelism for more details).

A `__global__` function must have void return type, and cannot be a member of a class.

Any call to a `__global__` function must specify its execution configuration as described in Execution Configuration.

A call to a `__global__` function is asynchronous, meaning it returns before the device has completed its execution.

7.1.2. __device__

The `__device__` execution space specifier declares a function that is:

- Executed on the device,
- Callable from the device only.

The `__global__` and `__device__` execution space specifiers cannot be used together.

7.1.3. __host__

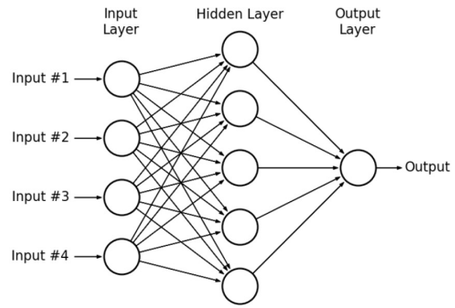
The `__host__` execution space specifier declares a function that is:

- Executed on the host,
- Callable from the host only.

CUDA MLP (2)

```
// Forward pass kernel for first layer (input -> hidden)
__global__ void layer1_forward(float *input, float *weights, float *bias, float *output, int batch_size) {
    int batch_idx = blockIdx.x * blockDim.x + threadIdx.x;
    int neuron_idx = blockIdx.y * blockDim.y + threadIdx.y;

    if (batch_idx < batch_size && neuron_idx < HIDDEN_SIZE) {
        float sum = bias[neuron_idx];
        for (int i = 0; i < INPUT_SIZE; i++) {
            sum += input[batch_idx * INPUT_SIZE + i] * weights[i * HIDDEN_SIZE + neuron_idx];
        }
        output[batch_idx * HIDDEN_SIZE + neuron_idx] = relu(sum);
    }
}

// Forward pass kernel for second layer (hidden -> output)
__global__ void layer2_forward(float *input, float *weights, float *bias, float *output, int batch_size) {
    int batch_idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (batch_idx < batch_size) {
        float sum = bias[0];
        for (int i = 0; i < HIDDEN_SIZE; i++) {
            sum += input[batch_idx * HIDDEN_SIZE + i] * weights[i];
        }
        output[batch_idx] = sum;  // No activation on output layer in this example
    }
}
```

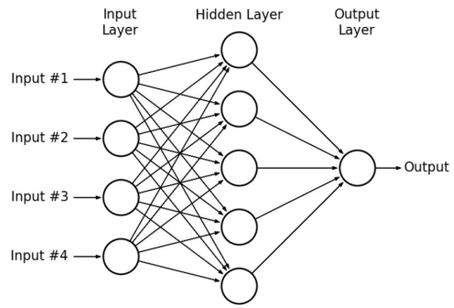
Global thread ID (x coordinate)
Global y coordinate



CUDA MLP (3)

Why separate kernels for each layer?

- **Memory optimization**: Each layer has different memory access patterns and working set sizes. Separate kernels allow you to optimize memory usage specifically for each layer's requirements rather than trying to find a one-size-fits-all approach.
- **Parallelism control**: Different layers may benefit from different thread configurations. For example, the first layer in our implementation handles 4 inputs to 5 neurons, while the output layer handles 5 inputs to 1 neuron. These naturally map to different parallelization strategies.
- **Resource utilization**: GPU resources like shared memory, registers, and thread counts can be tailored specifically to each layer's computational needs, which improves overall efficiency.
- **Kernel fusion limitations**: While fusing operations into a single kernel can reduce launch overhead, there are practical limits to how much computation can be packed into one kernel before it becomes inefficient. Separate kernels help manage this complexity.
- **Synchronization points**: Having distinct kernels creates natural synchronization points between layers, ensuring all computations from one layer are complete before the next layer begins processing.
- **Debugging and profiling**: With separate kernels, it's easier to identify performance bottlenecks or errors in specific parts of the network.
- **Specialized optimizations**: Different mathematical operations may benefit from specialized implementations - convolutions, fully-connected layers, and activation functions all have different optimal implementations on GPUs.

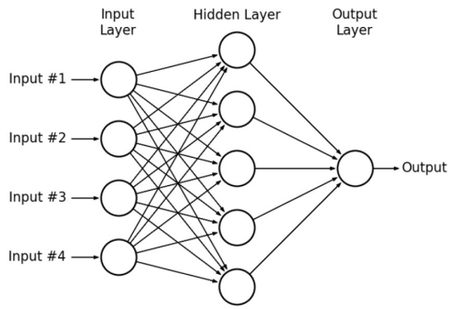


CUDA MLP (4)

How much should be in a kernel?

The optimal kernel size involves balancing several trade-offs:

- **Kernel launch overhead**: Each kernel launch incurs some overhead. Too many small kernels can result in performance degradation due to this overhead.
- **Register pressure**: Larger kernels with more operations typically require more registers per thread, which can reduce occupancy (the number of threads that can run simultaneously on the GPU).
- **Instruction cache**: GPUs have limited instruction cache. Very large kernels might not fit well in the cache, causing additional latency.
- **Memory bandwidth utilization**: Kernels that load data, perform a small operation, and then store results can be memory-bound. Combining multiple operations in one kernel can improve arithmetic intensity and make better use of compute resources.
- **Thread divergence**: If your kernel contains many conditional branches that cause different threads to follow different execution paths, performance will suffer.

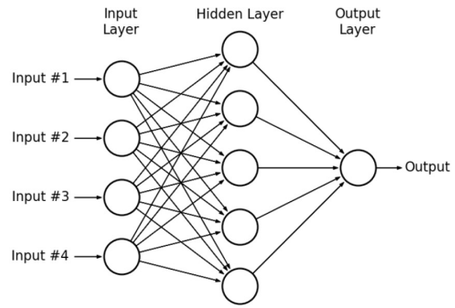


As a general guideline:

- **Good candidates for fusion**: Operations that work on the same data and have similar parallelization patterns (like a matrix multiplication followed by an element-wise activation function).
- **Better kept separate**: Operations with fundamentally different access patterns or parallelization strategies.

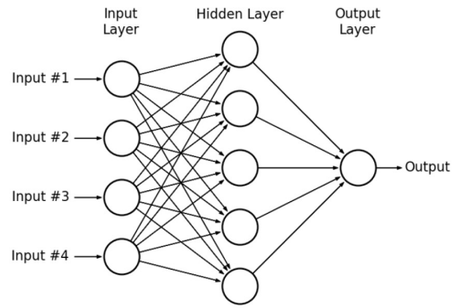
CUDA MLP (5)

```
// Initialize weights with random values between -0.5 and 0.5
void initialize_weights(float *h_weights, int size) {
    for (int i = 0; i < size; i++) {
        h_weights[i] = ((float)rand() / RAND_MAX) - 0.5f;
    }
}

int main() {
    // Seed random number generator
    srand(time(NULL));

    // Host memory allocation
    float h_input[BATCH_SIZE * INPUT_SIZE];
    float h_hidden[BATCH_SIZE * HIDDEN_SIZE];
    float h_output[BATCH_SIZE * OUTPUT_SIZE];
    float h_weights1[INPUT_SIZE * HIDDEN_SIZE];
    float h_bias1[HIDDEN_SIZE];
    float h_weights2[HIDDEN_SIZE * OUTPUT_SIZE];
    float h_bias2[OUTPUT_SIZE];

    // Initialize weights and biases
    initialize_weights(h_weights1, INPUT_SIZE * HIDDEN_SIZE);
    initialize_weights(h_bias1, HIDDEN_SIZE);
    initialize_weights(h_weights2, HIDDEN_SIZE * OUTPUT_SIZE);
    initialize_weights(h_bias2, OUTPUT_SIZE);

    // Initialize some example input data
    for (int i = 0; i < BATCH_SIZE * INPUT_SIZE; i++) {
        h_input[i] = ((float)rand() / RAND_MAX);
    }
```



CUDA MLP (6)

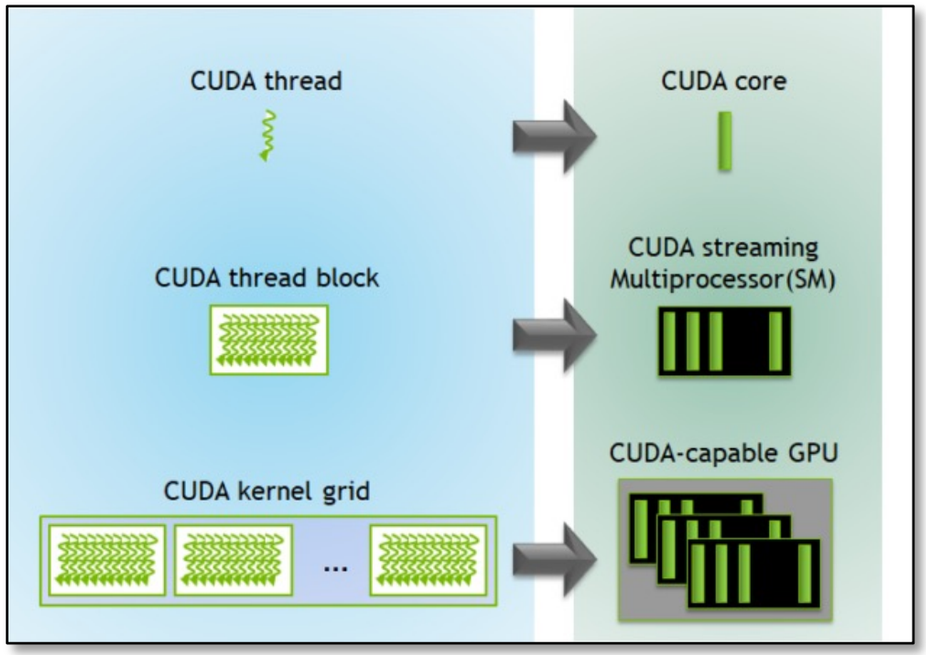
```
// Host memory allocation
float h_input[BATCH_SIZE * INPUT_SIZE];
float h_hidden[BATCH_SIZE * HIDDEN_SIZE];
float h_output[BATCH_SIZE * OUTPUT_SIZE];
float h_weights1[INPUT_SIZE * HIDDEN_SIZE];
float h_bias1[HIDDEN_SIZE];
float h_weights2[HIDDEN_SIZE * OUTPUT_SIZE];
float h_bias2[OUTPUT_SIZE];
```

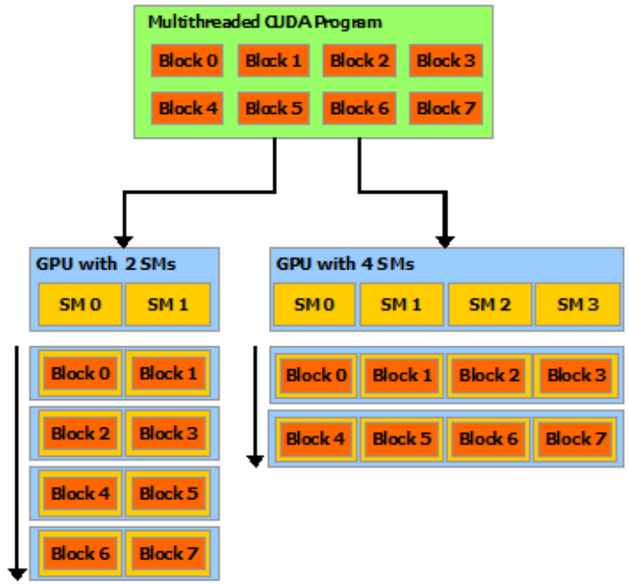


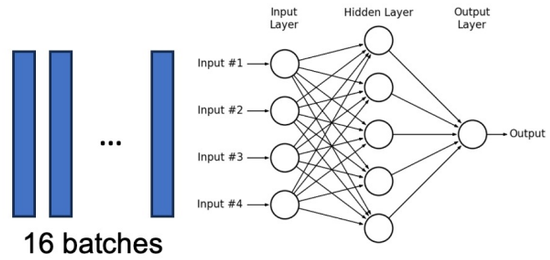

```
// Device memory allocation
float *d_input, *d_hidden, *d_output;
float *d_weights1, *d_bias1, *d_weights2, *d_bias2;

cudaMalloc(&d_input, BATCH_SIZE * INPUT_SIZE * sizeof(float));
cudaMalloc(&d_hidden, BATCH_SIZE * HIDDEN_SIZE * sizeof(float));
cudaMalloc(&d_output, BATCH_SIZE * OUTPUT_SIZE * sizeof(float));
cudaMalloc(&d_weights1, INPUT_SIZE * HIDDEN_SIZE * sizeof(float));
cudaMalloc(&d_bias1, HIDDEN_SIZE * sizeof(float));
cudaMalloc(&d_weights2, HIDDEN_SIZE * OUTPUT_SIZE * sizeof(float));
cudaMalloc(&d_bias2, OUTPUT_SIZE * sizeof(float));

// Copy data from host to device
cudaMemcpy(d_input, h_input, BATCH_SIZE * INPUT_SIZE * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_weights1, h_weights1, INPUT_SIZE * HIDDEN_SIZE * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_bias1, h_bias1, HIDDEN_SIZE * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_weights2, h_weights2, HIDDEN_SIZE * OUTPUT_SIZE * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_bias2, h_bias2, OUTPUT_SIZE * sizeof(float), cudaMemcpyHostToDevice);
```

CUDA MLP (7)

How will we use threads for this MLP?







A GPU is built around an **array of Streaming Multiprocessors (SMs). A multithreaded program is partitioned into blocks of threads that execute independently from each other**.

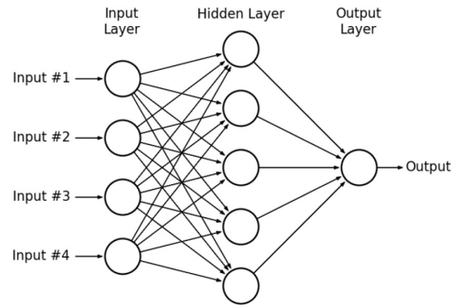
Christof Teuscher • teuscher@pdx.edu

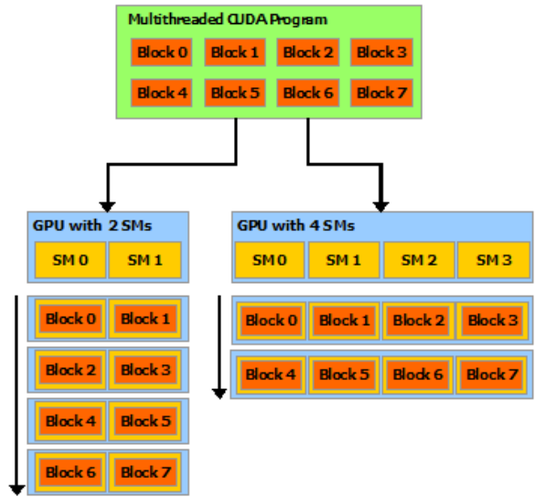
CUDA MLP (8)

```
// Set up execution configuration for first layer
dim3 threadsPerBlock1(8, 5);  // 8 batches, all 5 hidden neurons
dim3 blocksPerGrid1((BATCH_SIZE + threadsPerBlock1.x - 1) / threadsPerBlock1.x,
                    (HIDDEN_SIZE + threadsPerBlock1.y - 1) / threadsPerBlock1.y);

// Set up execution configuration for second layer
dim3 threadsPerBlock2(16, 1);  // 16 batches
dim3 blocksPerGrid2((BATCH_SIZE + threadsPerBlock2.x - 1) / threadsPerBlock2.x, 1);

// Launch kernels
layer1_forward<<<blocksPerGrid1, threadsPerBlock1>>>(d_input, d_weights1, d_bias1, d_hidden, BATCH_SIZE);
layer2_forward<<<blocksPerGrid2, threadsPerBlock2>>>(d_hidden, d_weights2, d_bias2, d_output, BATCH_SIZE);

// Check for errors
cudaError_t err = cudaGetLastError();
if (err != cudaSuccess) {
    printf("CUDA Error: %s\n", cudaGetErrorString(err));
    return -1;
}

// Copy results back to host
cudaMemcpy(h_output, d_output, BATCH_SIZE * OUTPUT_SIZE * sizeof(float), cudaMemcpyDeviceToHost);

// Print some outputs
printf("Sample outputs from MLP:\n");
for (int i = 0; i < 5; i++) {  // Print first 5 results
    printf("Sample %d: %.6f\n", i, h_output[i]);
}
```

Why?

`kernel<<M,T>>`
The kernel launches with a grid of M thread blocks.
Each thread block has T parallel threads.

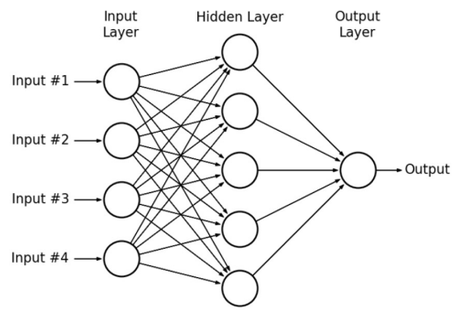




A GPU is built around an **array of Streaming Multiprocessors (SMs). A multithreaded program is partitioned into blocks of threads that execute independently from each other**.

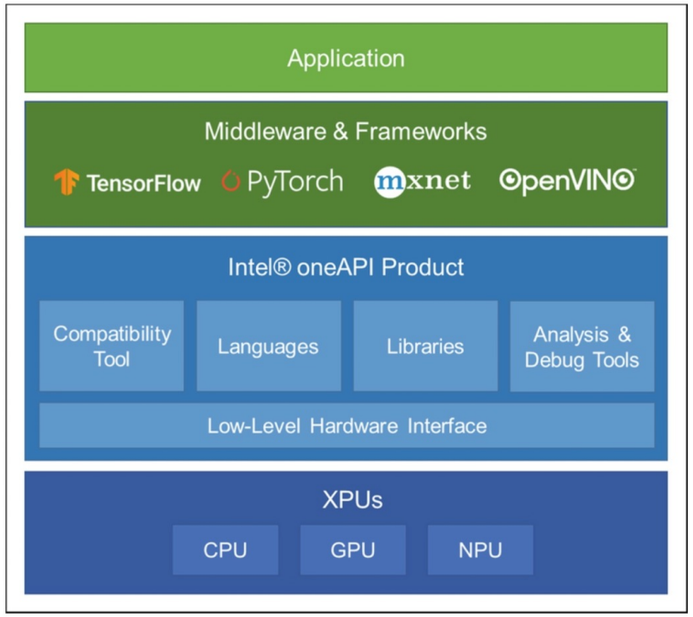
CUDA MLP (9)

```
    // Free GPU memory
    cudaFree(d_input);
    cudaFree(d_hidden);
    cudaFree(d_output);
    cudaFree(d_weights1);
    cudaFree(d_bias1);
    cudaFree(d_weights2);
    cudaFree(d_bias2);
```



Deep learning frameworks

1. Deep learning frameworks provide interfaces to implement AI functionalities regardless of hardware or computing platforms.
2. Most deep learning frameworks also optimize the network in a **hardware-friendly manner** and utilize the hardware resources and acceleration functions as much as possible.
3. Because GPUs are commonly used in deep learning, deep learning frameworks **widely support constructing and deploying neural networks on GPU** and actively reflect the acceleration support from GPU vendors.
4. In addition, they provide optimizations on standalone accelerators such as **TPU** and specific environments such as cloud servers.



TensorFlow

Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

PyTorch & CNNs

For CNNs, the mapping process involves:

1. **Tensor operations**: PyTorch represents all data (images, weights, activations) as tensors that can be moved between CPU and GPU memory.
2. **Layer-by-layer optimization**: Each CNN layer (convolution, pooling, etc.) is mapped to specific GPU kernels that efficiently perform the required operations in parallel.
3. **Memory management**: PyTorch handles the complex memory allocations and transfers between host (CPU) and device (GPU) memory.
4. **Computation graphs**: PyTorch builds a dynamic computation graph that tracks operations and enables automatic differentiation for training.
5. **Batching**: Multiple inputs are processed simultaneously to maximize GPU utilization.
6. For the **convolution operations** specifically, PyTorch typically uses highly optimized libraries like cuDNN that implement various algorithms (direct convolution, FFT-based, Winograd, etc.) and automatically select the most efficient one based on input size and available GPU resources.

Christof Teuscher • teuscher@pdx.edu

PyTorch

```python
import torch
import torch.nn as nn
import torch.optim as optim

# Define the network architecture
class SimpleMLPNet(nn.Module):
    def __init__(self):
        super(SimpleMLPNet, self).__init__()
        # First layer: 4 input neurons -> 5 hidden neurons
        self.fc1 = nn.Linear(4, 5)
        # Second layer: 5 hidden neurons -> 1 output neuron
        self.fc2 = nn.Linear(5, 1)
        # ReLU activation for hidden layer
        self.relu = nn.ReLU()

    def forward(self, x):
        # Forward pass through first layer and apply ReLU
        x = self.relu(self.fc1(x))
        # Forward pass through output layer (no activation)
        x = self.fc2(x)
        return x
```

```python
# Create a batch of random input data (16 samples, 4 features each)
batch_size = 16
input_size = 4
x = torch.rand(batch_size, input_size)

# Create the network
model = SimpleMLPNet()

# Move everything to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
x = x.to(device)

# Forward pass
with torch.no_grad():  # No need to track gradients for inference
    output = model(x)

# Print some sample outputs
print("Sample outputs from MLP:")
for i in range(5):  # Print first 5 results
    print(f"Sample {i}: {output[i].item():.6f}")
```

GPU vs TPU

GPU vs TPU

	GPU	TPU
Origin and design purpose	Graphics	Created by Google for ML
Architecture	Thousands of smaller cores designed for parallel processing.	More specialized architecture with matrix processing units (MXUs) specifically optimized for tensor operations.
Performance focus	More flexibility, broader applicability	Optimized for matrix operations and ML workloads.
Development and availability	NVIDIA, AMD, etc.	Proprietary, only through Google cloud
Software compatibility	Various frameworks, e.g., CUDA	TensorFlow
Power efficiency	Less power-efficient, but more flexibility	More power-efficient

Hardware for AI/ML

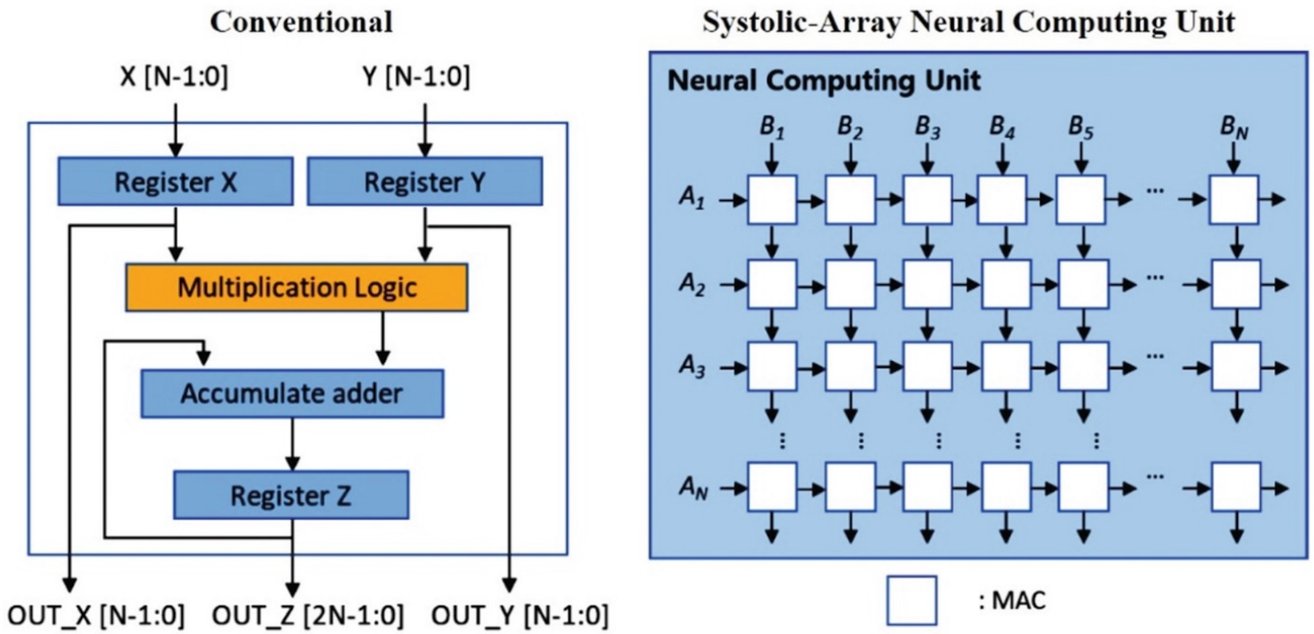


Fig. 19 A comparison of MAC operations performed on conventional and neural computing units.

Mishra et al., 2023

Christof Teuscher • teuscher@pdx.edu

www.teuscher-lab.com/teaching

Systolic array

WHAT IS A SYSTOLIC ARRAY?

A 2D grid of identical Processing Elements (PEs) that rhythmically compute and pass data to their neighbours, like a pulse.
Each PE performs one **Multiply-Accumulate** (MAC): multiply two inputs, add to a running sum, then forward data onward.

HOW DATA FLOWS (WEIGHT-STATIONARY)

Weights: preloaded into each PE and held fixed for the duration of the computation.
Activations: stream in from the left edge, one column per clock cycle, rippling rightward.
Partial sums: accumulate inside each PE row; final results drain out the right edge.

MATRIX MULTIPLY AS A MAC STREAM

C[i,j] = Σ A[i,k] × B[k,j] (summed over k)
Each PE computes one term of this sum per cycle, i.e., **the whole array fills C in parallel.**

Systolic arrays trade flexibility for throughput — hardwired MACs fire every cycle with near-zero memory traffic, making GEMM far more efficient than a general-purpose processor.

WHY IT MATTERS FOR DEEP LEARNING

No memory bottleneck: data moves PE→PE locally each cycle — no repeated reads from slow DRAM.
Massive parallelism: thousands of MACs fire simultaneously every clock cycle.
Energy efficient: minimal data movement means far lower power draw vs. a Von Neumann CPU or GPU.
High utilisation: every PE is active every cycle when matrices are large.

WHERE YOU SEE SYSTOLIC ARRAYS

Google TPU MXU: purpose-built large 2D systolic array for high-throughput GEMM.
NVIDIA Tensor Cores: Volta and later GPUs use systolic-inspired pipelines for mixed-precision matrix ops.
Apple Neural Engine / Huawei Ascend: on-device NPUs integrating systolic arrays for inference.

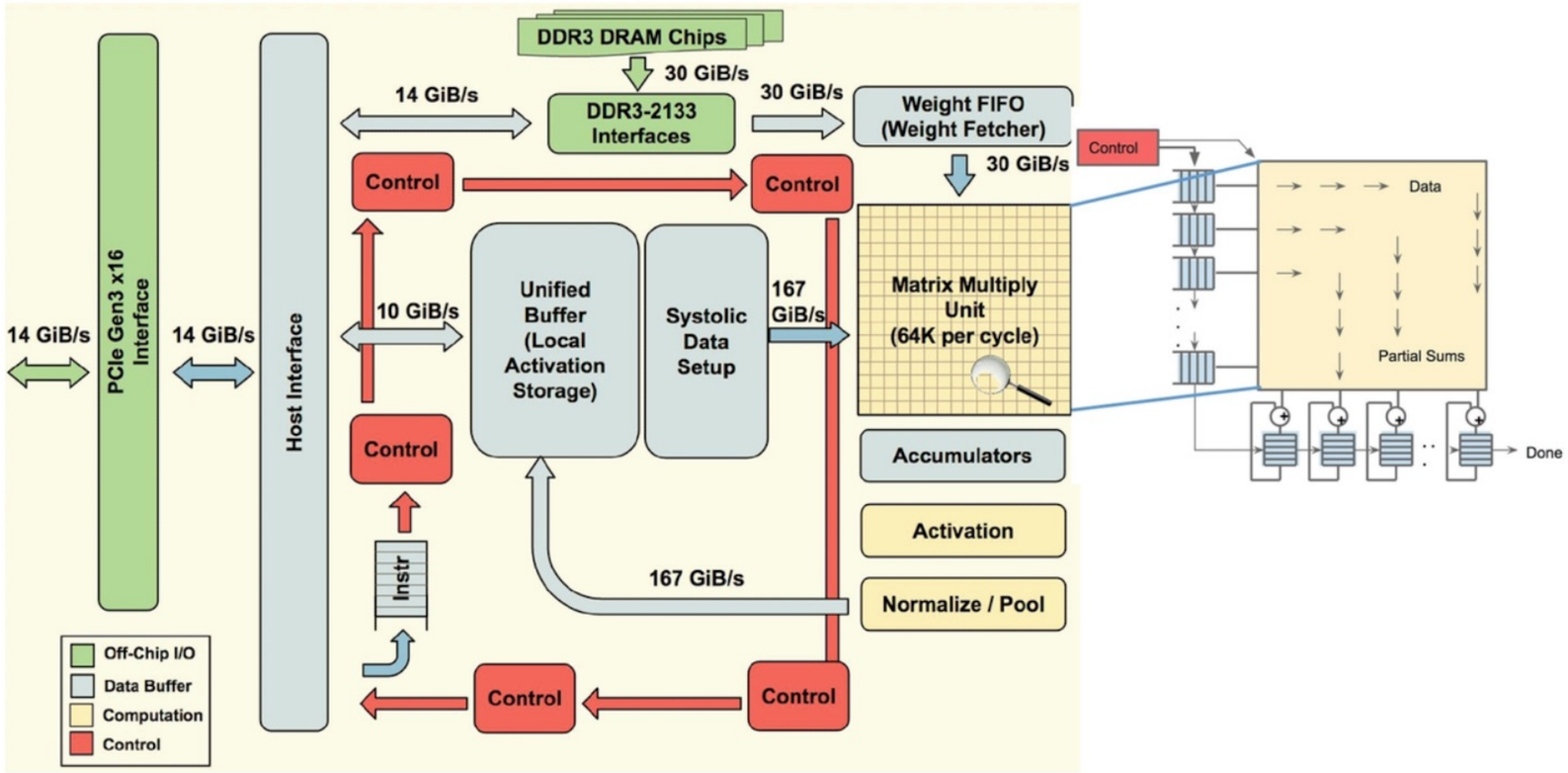
THREE DATA FLOW STRATEGIES

WS — Weight Stationary: weights held in PEs; activations & sums stream through. Best when weights are reused across many batches (TPU).
IS — Input Stationary: each PE holds one activation fixed; weights & partial sums stream in. Minimises activation reads — suits batch-size-1 inference.
OS — Output Stationary: each PE accumulates one full C[i,j] without passing it on. Weights & activations both stream through. Eliminates partial-sum traffic between PEs.

teuscher Lab
teuscher-lab.com

Portland State UNIVERSITY

Architecture of a TPU accelerator



- The main component of TPU is the MXU that consists of 256 by 256 (i.e., 65,536) MACs to perform 8-bit mathematical operations.
- The MXU gets its input from the weighted FIFO and unified buffer (UB).

Mishra et al., 2023